NewsInsights

Capstone Project Report
END SEMESTER EVALUATION

Submitted by:
(102216134) Nandan Garg

(102216081) Sakaar Sen

(102216019) Ridham Uppal

(102216032) Palak Mahajan

(102216103) Manav Singhal

BE Fourth Year, CoSE

CPG No: 92

Under the Mentorship of

Dr. Rinkle Rani

Professor

Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology, Patiala
December 2025

ABSTRACT

NewsInsights uses advanced Machine Learning and Natural Language Processing technology to provide its users with a news consumption experience that is bias-aware, personalized ,and credible. The platform automatically aggregates news across multiple different sources and provides users with concise AI-generated summaries, sentiment analysis, keyword analysis and political bias scoring for each news item so users can quickly identify both the content of the news story as well as its perspective.

The NewsInsights platform has demonstrated significant improvement over traditional news aggregators. Users are given the ability to compare news articles in real-time and based upon their source(s) using a vector-based recommendations model. This model decreases the potential for users to fall into "echo chambers" by encouraging users to explore a wider range of perspectives. Unlike static bias detection systems that use manually labeled data, NewsInsights utilizes adaptive machine learning models to identify, and classify new narratives, allowing users to receive a constant stream of accurate and timely bias evaluations.

This enables users to develop a more comprehensive understanding of media literacy, to become better at evaluating the credibility of news sources, and to gain a higher level of awareness regarding political trends and social issues. This project directly combats falsehoods and polarized reporting by creating an environment where appropriate, trustworthy news is available to support informed decision making in today's complex information ecosystem.

DECLARATION

We hereby declare that the design principles and working prototype model of the project entitled News Insights is an authentic record of our own work carried out in the Computer Science and Engineering Department, TIET, Patiala, under the guidance of Dr. Rinkle Rani during 6th and 7th semester (2025).

Date: 22 December 2025

Roll No.	**Name**	**Signature**
102216019	Ridham Uppal	*Ridham Uppal*
102216032	Palak Mahajan	*Palak Mahajan*
102216081	Sakaar Sen	*Sakaar Sen*
102216105	Manav Singhal	*Manav Singhal*
102216134	Nandan Garg	*Nandan Garg*

Counter Signed By:

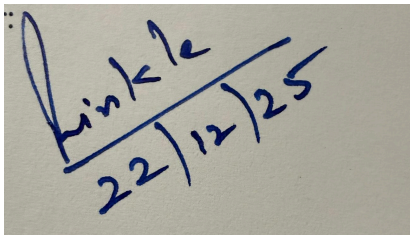
Faculty Mentor:

Dr. Rinkle Rani

Professor

DCSE,

TIET, Patiala



ACKNOWLEDGEMENT

We would like to express our thanks to our mentor Dr. Rinkle Rani. She has been of great help in our venture and an indispensable resource of technical knowledge.She is truly an amazing mentor to have.

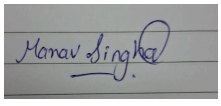
We are also thankful to Dr. Neeraj Kumar, Head, Computer Science and Engineering Department, the entire faculty and staff of the Computer Science and Engineering Department, and also our friends who devoted their valuable time and helped us in all possible ways towards successful completion of this project. We thank all those who have contributed either directly or indirectly towards this project.

Lastly, we would also like to thank our families for their unyielding love and encouragement.

They always wanted the best for us and we admire their determination and sacrifice.

Date: 22 December 2025

Roll No.	Name	Signature
102216134	Nandan Garg	*Nandan Garg*
102216081	Sakaar Sen	*Sakaar Sen*
102216019	Ridham Uppal	*Ridham Uppal*
102216032	Palak Mahajan	*Palak Mahajan*
102216103	Manav Singhal	*Manav Singhal*



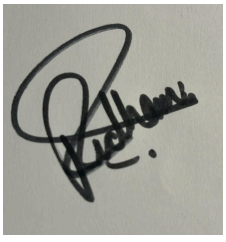
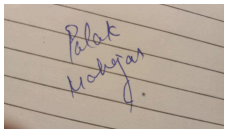
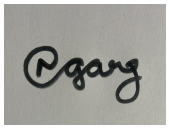




TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

ABBREVATIONS

Abbrevation	Meaning .
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
LLM	Large Language Model
BERT	Bidirectional Encoder Representations from Transformers
RoBERTa	Robustly Optimized BERT Approach
DeBERTa	Decoding-Enhanced BERT with Disentangled Attention
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
UI	User Interface
API	Application Programming Interface
DB	Database
ER	Entity–Relationship
RSS	Really Simple Syndication
WBS	Work Breakdown Structure
DRF	Django REST Framework

INTRODUCTION

1.1 Project Overview

1.1.1 Technical Terminology

AI- Artificial Intelligence refers to methods in computing that allow for machines to exhibit human-like capabilities in terms of reasoning and deciding as well as learning. In the case of this project, AI has been used to process and organize a large volume of news data so that it may be analyzed or summarized by human beings.

ML- Machine Learning (ML) is a specific category of AI which allows systems to recognize and understand relationships and patterns among data without having to write computer code for the purpose of identifying that relationship and/or understanding the pattern. In this project, we have used supervised and unsupervised ML to develop the means for classifying news sources according to political bias, clustering them according to those classifications, and making recommendations based upon their classifications.

NLP- Natural Language Processing is an area of AI that attempts to allow machines to be able to read, understand, interpret, and produce text written or spoken by humans in their language. In this project, NLP techniques have been applied to preprocess text, summarize text, perform sentiment analysis of the text, have a way to extract key words from the text, and also detect bias in the text.

Political Bias Detection is the process by which an ideology is assigned to a textual source (e.g., Left, Right, Neutral). We developed a method to automatically assess political bias through the use of the transformer models that we trained on our labeled political datasets.

Sentiment Analysis is the assessment of how a particular piece of textual data makes a person feel (positive, negative, neutral). In this project, sentiments are assessed so that we may better understand how news events are presented to readers emotionally.

Text Summarization is a process that shortens long forms of text into short summary formats, while still maintaining crucial content. Abstractive summarization models will be used in this project to assist users with information overload.

Embeddings are numerical representations of text that contain the meaning of the text in a compact form (dense). In this project, sentence embeddings will be used to determine the similarity between news articles and perform clustering and recommend articles to the user.

Clustering is a form of unsupervised learning that groups similar items of data together. In this project, we will utilize clustering to group news articles based on a similar topic. This can reduce duplicate articles, as well as help with comparative bias analysis.

Recommendation system is an algorithm that provides a user with content that they will be interested in based on their profile and previous interactions. In this project, the recommendation system will build upon user interactions and use similarity based on sentence embeddings to provide the user with personalized news recommendations.

REST (Representational State Transfer) is a type of architecture commonly used to develop networked applications. The backend services of the system expose RESTful APIs to enable communication between the frontend and backend of the application.

1.1.2 Problem Statement

In today's digital age, news consumption is at an all-time high, yet the landscape is increasingly fragmented. With countless media outlets, each carrying their own political leanings and editorial agendas, readers often struggle to identify unbiased and trustworthy information. Social media further amplifies this problem by prioritizing engagement over accuracy, resulting in echo chambers and misinformation. Against this backdrop, there is a growing demand for platforms that not only consolidate diverse news sources but also present them in a fair, transparent, and accessible manner.

The motivation behind developing NewsInsights stems from the need to bridge this gap between overwhelming information availability and meaningful, unbiased understanding. Many readers today face "information overload" with lengthy articles, repetitive coverage, and sensationalized headlines that make it difficult to quickly grasp the essence of current events.

Another key motivation is personalization without bias reinforcement. While most platforms push content aligned with existing user preferences, this often narrows perspectives. NewsInsights instead aims to provide readers with personalized yet diverse content by giving them relevant updates while ensuring they are not trapped in one-sided narratives. Additionally, features like bookmarks and sharing enhance the reading experience, making it not only informative but also convenient and user-friendly.

NewsInsights is an AI-powered unbiased news aggregator designed to address the challenges of media bias, information overload, and fragmented content consumption. The platform provides readers with a consolidated view of current events by collecting articles from multiple sources, analyzing their political bias, summarizing the content, and recommending personalized yet balanced information. Unlike conventional aggregators that merely display articles, NewsInsights emphasizes fairness, transparency, and clarity, enabling users to engage with news critically rather than passively.

At its core, the system leverages NLP and ML models to classify news into left, right, or neutral political orientations. This classification ensures that users are aware of the potential biases in reporting. In addition, the platform generates concise AI-driven summaries, reducing the time required to comprehend complex articles and making news more accessible to a wider audience.

Another defining aspect of NewsInsights is its multi-layered analytical capability. At its core, the platform not only detects political bias and generates concise summaries, but also incorporates sentiment analysis to reflect public opinion drawn from social media discussions. This helps readers understand how issues are perceived beyond traditional

reporting. Additionally, features like keyword highlighting support quick navigation through articles, giving users a clearer sense of the themes being discussed. By combining these layers, NewsInsights transforms raw information into structured insights, empowering readers to make informed judgments.

The system architecture follows a modular design, where the backend handles data ingestion, analysis, and processing, while the frontend ensures an interactive and seamless user experience. The platform's recommendation engine focuses on delivering a personalized reading experience by suggesting articles that match user interests while also prioritizing recent and unread content. This ensures that users stay updated with the latest developments without encountering repetitive information.

1.1.3 Goal

1. To aggregate news from multiple online sources.
2. To use AI for content summarization, sentiment analysis, and keyword extraction.
3. To implement a custom machine learning model to detect and classify political bias in articles.
4. To deliver personalized news recommendations based on user behavior.
5. To allow users to customize their news feed with filtering options as well as bookmark news articles.

Core Features

1. News Aggregation: Collects articles from multiple trusted sources.
2. Political Bias Detection: Machine learning model that classifies news into Left, Right, or Neutral categories.
3. AI Summaries: NLP-driven concise summaries of long articles for faster understanding.
4. Sentiment Analysis: Identifies the tone (positive, negative, or neutral) within news articles.
5. Recommendation Engine: Suggests articles tailored to user interests and prioritizes recent unread content.
6. Bookmarks: Enables users to save and revisit important articles for later reading.

Technologies Used

1. Backend: Django & Django REST Framework (API services)
2. Task Processing: Celery + Redis for background processing
3. Database: PostgreSQL for structured data storage. PgVector Extension to store vector embeddings.

4. Frontend: Next.js for user interaction and UI design
5. NLP/ML: Hugging Face transformers, custom ML models for bias classification, summarization, and sentiment analysis
6. Deployment: Docker for containerization.

1.1.4 Solution

NewsInsights is an AI-driven news aggregation platform designed to create an unbiased, transparent and user-centred experience, so that it achieves the stated goals of providing a single source of consolidated news articles sourced by multiple trusted sources. NewsInsights' mission is to convert unstructured news articles into a structured format providing useful insights that can be effortlessly consumed by consumers.

Using web scraping and real-time RSS feeds, NewsInsights aggregates the latest news articles from multiple outlets to ensure that the consumer can obtain a range of different types of news. NewsInsights uses an algorithm based on machine learning to detect the political bias of an article's content, and classify it as having left, right or neutral political bias. This allows consumers to become aware of the ideological framing of a news story, rather than simply consuming news passively.

To reduce the cognitive load on the consumer, NewsInsights employs natural language processing (NLP) based abstractive summarisation techniques to create concise yet informative summaries, preserving the most important contextual information.

Additionally, Articles in NewsInsights employ keyword extraction, along with entity highlighting to assist consumers in quickly identifying important topics, people or organisations within an article. Sentiment Analysis is performed on article content to provide consumers with a more complete view of how a news article has been framed by the news media.

To further promote engagement with the product, the NewsInsights structure also includes a personalized recommendation engine that uses interaction data and semantic similarity to provide consumers with recommendations for articles similar to those they have previously viewed.

1.2 Need Analysis

In today's digital landscape, news consumption has become increasingly complex due to bias, misinformation, and information overload. With numerous sources presenting the same news with different perspectives, users struggle to distinguish between factual reporting and biased narratives. Traditional news platforms often fail to provide a balanced view, leaving audiences vulnerable to echo chambers and manipulated information.

Individual news websites typically present events from a single perspective and do not offer multi-source comparisons. While existing aggregators like Google News and Flipboard do bring together articles from different outlets, they still lack political bias detection or real-time AI-driven analysis. Some platforms, such as Ground.News, attempt to address media bias but often depend on pre-labeled datasets instead of dynamic analysis. NewsInsights bridges these gaps by leveraging ML to analyze and summarize news from multiple perspectives, ensuring users gain a balanced and fact-based understanding of current events.

The relevance of this project in the real world extends beyond individual users to academics, journalists, and policymakers, who require unbiased, well-structured, and data-driven news insights. By integrating bias-aware analysis, keyword highlighting, and trend detection, NewsInsights promotes media literacy and critical thinking, helping users navigate the complexities of modern journalism. As misinformation continues to influence public opinion and decision-making, the need for an intelligent, fact-driven, and AI-powered news aggregator like NewsInsights has never been more critical.

1.3 Research Gaps

1. Interpretability and Explainability (XAI) Beyond Classification:
    a. Explanation: The dominant paradigm in bias detection remains classification—assigning a discrete label (e.g., "left," "right," "neutral") to a piece of text. While model accuracy has improved, this approach provides little insight into the nature of the detected bias. A simple label is insufficient for the ultimate goals of fostering media literacy, training journalists, or enabling nuanced academic analysis. The PRISM framework's move towards interpretable embeddings is a promising but nascent direction.
    b. Identified Gap: There is a critical need for models that move beyond classification to generation of fine-grained, evidence-based explanations. A truly useful system should not only flag a sentence as biased but also perform extractive explanation by highlighting the specific words or phrases contributing to the bias, and abstractive explanation by identifying the rhetorical or framing device being employed (e.g., "This sentence uses loaded language to frame the subject negatively" or "This passage creates a false balance by giving undue weight to a fringe viewpoint"). Research in this area is essential for building systems that are not just detectors but also diagnostic and educational tools.

2. Cross-Lingual and Cross-Cultural Bias Detection:
    a. Explanation: The existing body of research is overwhelmingly concentrated on the English language and the specific socio-political context of the United States. This creates a significant gap in our understanding, aBs political bias is a global phenomenon that manifests in culturally and linguistically specific ways. The framing of issues, the choice of loaded terms, and the entire political spectrum can differ dramatically between a two-party presidential system and a multi-party parliamentary one.
    b. Identified Gap: A major research frontier lies in the development of robust multilingual and cross-cultural bias detection systems. This requires a multi-pronged effort:
    c. Creating new, culturally-aware annotation schemas and datasets for a diverse range of languages and political systems;
    d. Investigating the efficacy of cross-lingual transfer learning to see if models trained on high-resource languages like English can be adapted to low-resource languages;
    e. Performing comparative analyses to understand how the linguistic markers of bias differ across cultures[16].
3. Modeling Subjectivity and Annotator Disagreement:
    a. Explanation: A fundamental flaw in the standard supervised learning approach is the assumption of a single, objective "ground truth" label for bias. In reality, the perception of bias is highly subjective. Forcing annotators to agree on a single label for an ambiguous text either results in low inter-annotator agreement (making the data unreliable) or washes out the very nuance the model should be learning. The BiasLab dataset is a notable exception that explicitly captures annotator perception and disagreement.
    b. Identified Gap: There is a significant opportunity to develop models that embrace subjectivity rather than ignoring it. Instead of predicting a single class, future models could be designed to predict a distribution of perceived bias. For example, a model could be trained to predict that "70% of liberal readers would perceive this sentence as neutral, while 80% of conservative readers would perceive it as biased." This would require new datasets annotated by a diverse panel of raters with recorded demographic and ideological information, as well as novel probabilistic model architectures capable of learning from and representing human disagreement[17].
4. Holistic Multimodal Bias Detection:
    a. Explanation: News consumption in the digital age is an inherently multimodal experience. The overall message and its potential bias are conveyed not just through the text of an article but through the interplay of headlines, images, captions, infographics, and embedded videos. An unflattering photograph of a politician, the selective cropping of an image to alter its context, or the juxtaposition of a neutral headline with a highly emotional image are all powerful methods of conveying bias that text-only models are blind to.
    b. Identified Gap: A largely unexplored research area is the development of models that can perform holistic, multimodal bias detection. This requires integrating state-of-the-art computer vision and NLP techniques to analyze how textual and

visual elements work together to construct a biased narrative. While initial datasets like NewsMediaBias-Plus provide a starting point , the architectural and theoretical frameworks for fusing and reasoning over this multimodal information are still in their infancy[18].

5. Robustness, Nuance, and Mitigating LLM-Specific Failures:
    a. Explanation: The increasing reliance on LLMs for bias detection introduces a new class of challenges related to their specific failure modes. These models are known to be highly sensitive to prompt phrasing and can exhibit sycophancy, where they uncritically adopt the biases present in the user's query.43 Furthermore, all current models, including LLMs, struggle with the most subtle and powerful forms of bias, particularly

    bias by omission, where the partiality is defined by what is strategically left unsaid. Detecting this requires a model to have deep world knowledge and the ability to reason about what constitutes a complete and balanced account of an event.
    b. Identified Gap: There is a pressing need for research focused on improving the robustness and reliability of LLMs for this sensitive task. This includes: (1) developing methods to quantify and mitigate the inherent political biases of the LLMs themselves 30; (2) designing adversarial training regimes and prompting strategies to make models less susceptible to manipulation and sycophancy; and (3) creating novel architectures and training objectives specifically designed to tackle non-lexical biases like omission, perhaps by incorporating knowledge graphs or cross-referencing multiple sources to identify missing information.

1.4 Problem Definition and Scope

The exponential rise in online news platforms has created an environment where readers are overwhelmed with diverse perspectives, hidden biases, and excessive redundancy of information. Traditional news aggregators mainly focus on presenting headlines from multiple sources but lack advanced mechanisms to detect bias, summarize content, or provide contextual clustering. This creates challenges in enabling readers to consume news in a balanced, concise, and meaningful way. Thus, there is a pressing need for an intelligent system that not only aggregates news but also enhances it through ML techniques.

The scope of this project is limited to the domain of text-based news aggregation and analysis. The system focuses on functionalities such as political bias detection, clustering of related articles, summarization, sentiment analysis, and recommendation of relevant content. The solution is designed to assist readers, researchers, and decision-makers in obtaining unbiased and concise news insights. However, the scope does not extend to real-time verification of authenticity of news, detection of deepfakes, or handling of multimedia formats such as images, audio, or video.

1.5 Assumptions and Constraints

Table 1: Assumptions and Constraints Table

S No.	Assumptions and Constraints
1.	It is assumed that the news data being aggregated originates from legitimate, trustworthy, and credible publishers. The system primarily collects content through scraping official websites or consuming RSS feeds. While the framework leverages the reputation of these sources to ensure reliability, it does not implement a dedicated fact-checking mechanism at the article level. Consequently, if misinformation exists at the source, such inaccuracies may be propagated within the aggregated system. This limits the platform's ability to guarantee absolute authenticity of the presented information.
2.	The framework assumes continuous accessibility to APIs, RSS feeds, and scraping endpoints from major news outlets to ensure multi-source data collection. These feeds are expected to provide comprehensive coverage of diverse domains such as politics, business, technology, sports, and entertainment. In scenarios where feeds become unavailable, are paywalled, or restrict automated access, the system's ability to compare across multiple sources and maintain balanced aggregation could be constrained. The system therefore operates under the assumption of long-term data availability from external publishers.
3.	The system assumes that the employed machine learning and natural language processing models for tasks such as content summarization and sentiment classification operate at an acceptable level of accuracy. While state-of-the-art pre-trained models are fine-tuned for these purposes, no AI system can achieve perfect precision or recall across all contexts, languages, and domains. Factors such as sarcasm, implicit bias, or culturally nuanced reporting may affect the accuracy of predictions, and occasional misclassifications must be acknowledged as a constraint of the system.
4.	It is assumed that user activity, such as article clicks, reading duration, reactions, or category selections, reliably represents genuine user preferences. The system interprets these behavioral signals as proxies for interests and incorporates them into the recommendation engine. However, user interactions may not always reflect true intent for example, a user might click on sensational content out of curiosity rather than preference, or avoid certain articles despite being interested due to lack of time. Such discrepancies may affect the personalization outcomes, limiting the accuracy of inferred interests.
5.	It is recognized that new users, lacking historical activity or preference data, may initially receive less accurate or generalized recommendations. The recommendation engine relies on accumulating sufficient interaction signals to progressively refine personalization. Similarly, newly published articles may lack adequate contextual embeddings or comparative clustering information during

	their initial inclusion in the database, leading to suboptimal categorization or recommendation. This “cold-start” limitation is intrinsic to recommendation systems and is mitigated gradually as user activity and article metadata grow over time.

1.6 Standards

This project aims to adhere to existing standards in the areas of software engineering, web development, and data security. In maintaining established software engineering standards, this will allow the project to be reliable and have scalability, interoperability, and be ethically compliant.

Software Engineering: The project will use the process defined by the Software Development Life Cycle (SDLC) which involves requirement analysis, system design, implementation, testing, and documentation. Additionally, the use of a modular architecture and the implementation of the separation of concerns will support the future maintainability of the project, as well as allow for the possibility of future expansion (scalability) of the project.

IEEE Documentation Standards: All technical reports produced by the project will follow the formatting and citation standards recommended by the IEEE for both academic and professional documentation. All of the references that will be located at the end of this report will follow IEEE citation format to ensure consistency and credibility of all references.

Web Standards and API Design: All backend service designs for this project will follow the RESTful API design standard which includes stateless communication, the use of HTTP methods (GET, POST, PUT, DELETE) correctly, and the use of a standardized response format. All services will use JSON as the primary format for exchanging data.

Database Data Management Standards The database design principles for the project will be developed using a relational database design approach with PostgreSQL to provide data integrity, normalization, and efficient querying of data. Vector Embedding will be stored and queried in accordance with industry standards for similarity search.

Security Standards Security best practices will be in place including role-based authentication, secure password storage, and communication based on HTTPS. Access to APIs will be restricted to prevent unauthorized usage, and will ensure that user data privacy will be of utmost importance.

Artificial intelligence (AI) must be used safely. The opportunity for individuals to understand how information is created, manipulated, and presented to them is an essential aspect of Using Artificial Intelligence for good.

Performance and Reliability Standards: The project has been developed to enhance the efficiency of the platform by utilizing concurrent connections and providing asynchronous processing capabilities. The project utilizes asynchronous queues and caching mechanisms to ensure prompt and responsive performance in times of high demand.

1.7 Approved Objectives

1. Efficient News Aggregation
   Develop a system that collects news articles from multiple sources in real-time, ensuring users receive the latest and most relevant updates.

2. AI-Powered Summarization, Sentiment Analysis & Keyword Extraction
   Utilize AI-powered summarization, sentiment analysis, and keyword extraction to generate concise summaries, highlight essential topics, and provide deeper insights for better content understanding.

3. Political Bias Detection.
   Design and integrate a custom machine learning model to analyze news articles and detect political bias and provide transparency in news reporting, enabling users to assess different viewpoints before forming opinions.

4. Personalized News Recommendations.
   Develop an AI-powered recommendation engine that suggests articles based on user preferences, reading history, and content relevance.

5. Content Filtering & Customization.
   Enable users to filter news content based on categories, sources, bias levels, or sentiment, ensuring a personalized experience.

1.8 Methodology

The methodology adopted for the NewsInsights project follows an iterative and incremental development model. This approach was chosen to effectively manage the complexities of integrating web development, data engineering, and machine learning components. The project lifecycle is broken down into five distinct phases, allowing for continuous refinement and integration.

1. System Design and Planning
   This initial phase focused on establishing the foundational blueprint for the project. It involved defining the system architecture, designing the database schema for PostgreSQL, and creating detailed API specifications. A comprehensive Work Breakdown Structure

(WBS) was developed to outline all tasks and deliverables, ensuring a clear development roadmap.

2. Data Aggregation and Processing Pipeline
   The second phase concentrated on building the core data ingestion engine. This involved developing robust web scraping scripts to collect news articles from diverse sources, implementing data cleaning and deduplication logic, and creating a process to generate and store vector embeddings for all content using SentenceTransformer models.

3. AI Model Development and Integration
   This phase involved the research, development, and integration of the core AI modules. It included implementing transformer-based models from Hugging Face like RoBERTa for sentiment analysis and Mistral for summarization, as well as training and deploying a custom machine learning classifier for political bias detection.

4. Full-Stack Application Development
   With the data and AI pipelines in place, this phase focused on building the user-facing application. The backend was developed using Django and Django REST Framework to create APIs for serving news, managing users, and delivering recommendations. Concurrently, the frontend was built as a single-page application using Next.js to create an interactive and responsive user interface.

5. System Integration, Testing, and Deployment
   The final phase involved integrating all components like the data pipeline, AI modules, backend, and frontend into a cohesive system. A comprehensive testing strategy, including unit, integration, and performance testing, was executed to ensure the platform's stability and reliability before preparing for containerized deployment with Docker.

1.9 Project outcomes and Deliverables

The proposed system is expected to achieve the following outcomes:

1. Aggregated News Feed: A unified news repository created by scraping and storing articles from multiple news outlets, enabling centralized access to diverse sources.
2. Clustered and Summarized Content: Automatic grouping of semantically similar news articles into clusters, with concise AI-generated summaries to reduce redundancy.
3. Sentiment and Political Bias Analysis: Identification of article-level sentiment (positive, negative, neutral) and political orientation (left, right, neutral), providing users with contextual insights.
4. Personalized Recommendations: AI-driven recommendations based on user interaction history, embeddings, and semantic similarity, ensuring relevance and personalization.

The key deliverables of this project include:

1. Working Web Application:
   Frontend interface for browsing, searching, and interacting with news articles.
   Backend services for scraping, clustering, sentiment analysis, summarization, and recommendations.

2. AI/ML Models:
   Custom built model for political bias detection.
   Recommendation engine based on embeddings and cosine similarity.

3. Documentation:
   IEEE-style project report with methodology, results, and analysis.
   User manual for application usage.

1.10 Novelty of Work

1. Transparent and Explicit Bias Detection
   A core innovation of this project is the integration of a custom-trained machine learning model to explicitly classify the political leaning of news content. While other platforms may implicitly try to balance sources, NewsInsights provides direct, transparent labeling as part of the user experience. This feature is a direct response to the growing problems of media echo chambers and misinformation, aiming to empower users with the tools for critical media consumption.

2. Comprehensive Aggregation for a Broader Perspective
   This ensures a complete and multi-faceted understanding of world events, a feat impossible for any single news channel due to inherent limitations in editorial focus, resources, and perspective. By systematically collecting articles from a wide array of sources spanning different political leanings, and categories the system allows users to see beyond a single narrative. This approach not only builds a more nuanced picture of major events by contrasting different angles.

3. Smart Personalization
   The recommendation system goes beyond simply showing trending topics. It combines recent news with user preferences using semantic embeddings and interaction data, ensuring that recommendations are both timely and user-centric.

2.1 Literature Survey

2.1.1 Related Work

Defining Bias is a deviation from standards of objective journalism that appears more often as subtle partisan slant than outright fabrication. It manifests as (a) content-level biases (omission, story selection, placement, source selection, labeling), (b) structural biases (gatekeeping, agenda-setting, framing), and (c) economic/psychological biases (corporate and demand-driven incentives; audience confirmation bias) [1].

1. Foundational communication theories:
    a. Agenda-Setting: Media influence what the public thinks about by elevating issue salience through selection and prominence [2].
    b. Framing: Media influence how the public thinks about issues by emphasizing selected aspects to define problems, diagnose causes, make moral judgments, and suggest remedies [3].

2. Operationalizing bias for computation:
    a. Transition from outlet-level analysis → lexical/sentence-level markers → model-level statistical properties.
    b. Two key linguistic forms from NPOV studies: framing bias (loaded/subjective language) and epistemological bias (presuppositions/entailments via grammar/verbs like “acknowledged” vs “claimed”).
    c. Modern view recognizes bias in model representations (representational and allocational harms) [4].
    d. Recursive challenge: Using LLMs to annotate bias risks laundering their intrinsic leanings into “ground truth,” creating feedback loops.

Existing Systems and Solutions

Phase 1 – Lexicon-based/statistical counts
Transparent but brittle; fail on context, negation, irony; struggle with evolving political language [5].

Phase 2 – Traditional ML (SVM/LogReg)
Better than lexicons but reliant on heavy feature engineering (n-grams, POS, syntax, sentiment)[6].

Phase 3 – Deep learning

1. RNN/LSTM: Capture order/short-range dependencies; data-hungry; limited long-range context.

2. Transformers (BERT family): Pretraining + self-attention yields contextual understanding; fine-tuning became standard; domain-adaptive pretraining (e.g., POLITICS on news corpora) boosts results[7]

Phase 4 – State of the art & LLMs:

1. Architectures beyond vanilla Transformers (graph-based for event/entity relations; cross-encoders for nuanced scoring).
2. LLMs (GPT, Llama, Gemini): Zero-/few-shot with advanced prompting (e.g., Chain-of-Thought).
3. Emerging emphasis on interpretability (e.g., PRISM embeddings, tools like BiasScanner) to answer not just "is it biased?" but "why and where?"

Table 2: Literature Survey

S. No.	Roll Number	Name	Paper Title	Tools/Technology	Findings	Citation
1	102216103	Manav Singhal	Target-Aware Contextual Political Bias Detection in News	BERT, RoBERTa, BASIL dataset, Data Augmentation (BANC, ABTA, EBTA)	Context is critical for sentence-level bias detection. Proposed target-aware data augmentation techniques significantly improve performance, achieving a state-of-the-art F1-score of 58.15 on the BASIL dataset. Highlights the difficulty of distinguishing informational vs. lexical bias.	Maab, I., Marrese-Taylor, E., & Matsuo, Y. (2023). In Proceedings of the 13th IJCNLP and the 3rd AACL (Volume 1: Long Papers).[9]
2			Political Leaning and Politicalness Classification of Texts	Transformer Models (BERT, RoBERTa, DeBERTa), POLITICS model, 12 leaning datasets, 18 politicalness datasets	Existing models suffer from poor generalization ('siloed solutions'). Introduces 'politicalness' as a prerequisite classification task. Creates a large, diverse dataset for this task. Finds that domain-specific pre-training (POLITICS model) is highly effective.	Volf, M., & Simko, J. (2025). arXiv:2507.13913[7].

3			Quantifying Generative Media Bias with a Corpus of Real-world and Generated News Articles	LLMs (GPT series, Llama 2, Mistral), RoBERTa, POLITICS model, AllSides dataset	Instruction-tuned LLMs exhibit consistent, measurable political bias (typically left-leaning). Proposes a metric for 'political bias shift' between human and AI text. Bias is topic-dependent and influenced by prompt design.	Trhlik, F., & Stenetorp, P. (2024). In Findings of the Association for Computational Linguistics: EMNLP 2024[10].
4			PRISM: A Framework for Producing Interpretable Political Bias Embeddings with Political-Aware Cross-Encoder	Cross-Encoder (DeBERTa-v3-large), LLMs (GPT-4o), NewsSpectrum & BigNews datasets	Proposes a shift from classification to producing interpretable embeddings. PRISM outperforms SOTA models in classification and diversified retrieval without fine-grained annotations. Embeddings are explicitly tied to controversial topics, enabling explainability.	Sun, Y., Huang, Q., Tung, A. K. H., & Yu, J. (2025). In Proceedings of the 63rd Annual Meeting of the ACL[8].
5			A Novel BERT-based Classifier to Detect Political Leaning of YouTube Videos based on their Titles	BERT, Word2Vec, GloVe, YouTube API, Custom dataset of 10M video titles	Adapts bias detection to a new domain (YouTube titles). Fine-tuned BERT model achieves 75% accuracy and 77% F1-score, outperforming alternatives. Demonstrates that even short, informal text like video titles contains strong political signals.	AlDahoul, N., Rahwan, T., & Zaki, Y. (2024). arXiv:2404.04261[11].
6			Navigating Nuance: In Quest for Political Truth	Llama-3 (70B), Chain-of-Thought (CoT) prompting, Media Bias Identification Benchmark (MBIB) dataset	Explores the effectiveness of advanced prompting (CoT) for LLMs in bias detection. Achieves performance comparable to a fully fine-tuned SOTA ConvBERT model, highlighting the power of in-context learning and reducing the need for extensive fine-tuning.	Sar, S., & Roy, D. (2024). In The 2024 ACM/IEEE Joint Conference on Digital Libraries (JCDL '24)[12].

2.1.2 Research Gaps for Existing Literature

1. Performance & difficulty:
   Transformers dominate, yet sentence-level bias detection remains hard (e.g., mid-50s F1 on BASIL), highlighting persistent ambiguity and nuance[9].

2. Primacy of context:
   Strong gains come from incorporating target-aware and cross-document context (multiple articles on the same event), discourse structure, and event graphs; bias is rarely visible from isolated tokens[10].

3. Granularity shift:
   Movement from outlet/article labels to sentence/span-level detection enables actionable highlights of biased language, though it is more challenging[7].

4. Data bottleneck:
   High-quality, fine-grained annotations are scarce and subjective; progress relies on data-centric strategies: unifying many datasets (e.g., "politicalness" as a precursor task), domain pretraining, and data augmentation tailored to events/targets[11].

5. LLM observations:
   Instruction-tuned LLMs show measurable, topic-dependent leanings; prompt design impacts outputs, underscoring the need for robustness and auditing[12].

2.1.3 Detailed Problem Analysis

1. Inherent subjectivity & low agreement:
   Bias judgments vary by reader ideology and background; omission bias is especially hard (reasoning about what's not said)[13].

2. Poor generalization/domain dependence:
   Models trained on U.S. mainstream news falter on social media, YouTube titles, opinion blogs, or other countries' media—yielding siloed solutions[14].

3. Data scarcity & noisy labels:
   Expert sentence-level labels are costly; weak supervision via publisher slant introduces noise and overfitting risks[15].

LLM-specific challenges

1. Inherent leanings from pretraining data.
2. Prompt sensitivity & sycophancy (alignment with user premise).
3. Opacity of decisions (black-box reasoning), motivating XAI approaches

4. Anglophone/U.S.-centric focus: Limited multilingual/cross-cultural coverage reduces global applicability[14].

2.1.4 Survey of Tools and Technologies Used

Benchmark datasets

1. BASIL: Sentence/span-level annotations with targets and bias types; cornerstone for fine-grained evaluation.
2. BABE: Expert sentence-level labels; used in practical explainers (e.g., BiasScanner).
3. MBIB: Unified benchmark across political/cognitive/linguistic bias types.
4. Publisher-level corpora (AllSides, BigNews, NewsSpectrum): Scalable but weak/noisy labels; valuable for pretraining and trend learning.
5. Emerging sets: BiasLab (captures annotator disagreement and rationales); NewsMediaBias-Plus (multimodal: text + images); domain/platform-specific corpora (social media, podcasts, YouTube)[15].

Core architectures/models

1. Transformer encoders: BERT/RoBERTa/DeBERTa as baselines for fine-tuning.
2. Domain-specific models: e.g., POLITICS (RoBERTa continually pre-trained on millions of news articles).
3. LLMs: GPT/Llama/Gemini for zero-/few-shot with advanced prompting.
4. Specialized designs: Cross-encoders for interpretable multi-dimensional scores (PRISM); Graph Neural Networks for event/entity relation reasoning

Ecosystem & frameworks

1. Hugging Face Transformers (models, datasets, training utilities).
2. PyTorch / TensorFlow as primary deep learning backends.

Key research gaps

1. Interpretability beyond labels: Extractive highlights + abstractive rhetorical explanations.
2. Cross-lingual/cross-cultural bias modeling and datasets.
3. Modeling subjectivity: Predicting distributions of perceived bias by audience segment.
4. Multimodal integration: Jointly analyzing headlines, body text, images, captions, and visuals.
5. LLM robustness: Bias auditing/mitigation, anti-sycophancy prompting/training, and strategies for detecting bias by omission (e.g., knowledge graphs, cross-source corroboration)

2.1.5 Summary

Our proposed work is firmly grounded in the current state-of-the-art for political bias detection, while introducing innovations in both contextual modeling and system architecture

We deliberately leverage proven techniques that have become the gold standard in computational linguistics:

1. Foundation in Transformer Models:
   For the core classification task, we rely on RoBERTa and DeBERTa, following the evidence that transformer-based models consistently outperform earlier deep learning methods such as LSTMs in capturing subtle linguistic markers of bias (Kulkarni et al., 2021; Chen et al., 2023). This ensures our system starts with a high-performing baseline aligned with best practices.

2. Leveraging Contextual Embeddings:
   To capture semantic similarity between news articles, we employ Sentence-BERT. Bias is inherently contextual, and shallow lexical features are insufficient. By embedding full article text (title + content), the system can reason about semantic framing choices rather than word overlap alone.

3. Data-Centric Strategies:
   We incorporate insights from prior research emphasizing the role of dataset unification, augmentation, and domain-adaptive pretraining. Our pipeline leverages these strategies to reduce dependence on limited annotated corpora and improve robustness.

4. Alignment with Explainable AI Trends:
   By extending beyond label prediction, our design integrates event comparisons and contextual analysis, aligning with ongoing efforts in interpretable bias detection frameworks such as PRISM.

At the same time, our methodology introduces several technical and conceptual innovations:

1. Focus on the Indian Political Context:

   Most political bias datasets and models (e.g., BASIL, AllSides) are U.S.-centric, overlooking linguistic and rhetorical nuances of Indian politics. By targeting Indian news sources, even in English, our system addresses this cultural and regional gap. Markers of bias—choice of descriptors, references to parties, and framing of events—are India-specific, and our work is among the first to model them systematically.

2. Novel Event-Clustering Pipeline:

   Unlike prior work that classifies articles in isolation, we introduce a semantic event-clustering stage before classification. This is implemented through:
   a. Sentence-BERT embeddings to represent articles.
   b. Deduplication via text hashing to prevent repeated content from skewing clusters.
   c. Temporal filtering (72-hour sliding window) to ensure articles grouped together are from the same event cycle.

d. Category-aware similarity boosts so coverage of the same type of event (e.g., elections, protests) clusters more tightly.
e. Dynamic centroid recomputation as new articles are added, keeping clusters semantically stable[10].
f. Database persistence (RawNewsFeed and Cluster models) to maintain embeddings and centroids across sessions for robustness.

This allows the model to detect framing bias and omission bias, which only emerge by comparing coverage across outlets reporting on the same event—something single-article models cannot achieve.

3. Hybrid Unsupervised + Supervised Design:

Our architecture combines unsupervised clustering with supervised classification:

a. Unsupervised Stage: Articles are organized into semantically coherent clusters of events using Sentence-BERT and temporal similarity checks.
b. Supervised Stage: Once clustered, a fine-tuned classifier (RoBERTa/DeBERTa) labels the clusters and their member articles with bias categories.

This hybrid setup makes the system:

a. Robust to noisy, unlabeled news corpora (since clustering organizes chaos into structure).
b. Scalable to large real-world news streams.
c. Context-aware, because bias is determined relative to how multiple outlets frame the same event, not how one article reads in isolation.

2.2 Software Requirement Specification

2.2.1 Introduction

2.2.1.1 Purpose

The purpose of this project is to design and implement an AI-powered news aggregator system that collects, processes, and presents news articles from multiple sources in a structured, unbiased, and user-personalized manner. The system aims to overcome limitations of traditional aggregators by integrating natural language processing (NLP) techniques for bias detection, summarization, sentiment analysis, and clustering of similar stories. Ultimately, the platform empowers users to access diverse perspectives, make informed judgments, and discover relevant news efficiently.

2.2.1.2 Intended Audience and Reading Suggestions

Developers & Engineers: To understand functional modules (scraping, clustering, recommendation) and technical specifications.
Researchers & Academics: To analyze novelty in bias detection and multilingual handling.

End Users: To explore system capabilities like recommendations, summaries, and unbiased reporting.

2.2.1.3 Project Scope

The system aggregates news from at least 10 major news outlets through web scraping and APIs, stores raw data in a structured database, and processes it using clustering and embeddings. NLP models then perform summarization, sentiment analysis, and bias classification to present concise and balanced perspectives. A recommendation engine tailors news feeds based on user activity and preferences.

The scope includes:

1. Multi-source data ingestion.
2. News clustering & redundancy reduction.
3. AI-powered summaries & bias detection.
4. Sentiment classification for articles.
5. Personalized recommendations.
6. Political Bias Detection
7. Secure and scalable web application for users.

2.2.2 Overall Description

2.2.2.1 Product Perspective

The system operates as a web-based platform where backend services (scraping, clustering, ML models) interact with a PostgreSQL database and vector embeddings for semantic similarity. The architecture follows a modular design.

2.2.2.2 Product Features

1. Aggregation of news from multiple sources.
2. Automatic clustering of similar articles to reduce redundancy.
3. AI-powered summarization for concise news delivery.
4. Political bias detection (Left, Right, Neutral).
5. Sentiment classification (Positive, Negative, Neutral).
6. Ability to add bookmarks.
7. Personalized recommendations based on user interaction.
8. Search and filter functionalities.

2.2.3 External Interface Requirements

2.2.3.1 User Interfaces

Web application (React-based frontend) with features like:

1. News cards with summaries, bias tags, and sentiment labels.
2. Filtering options by category, sentiment, or bias.
3. Recommendation section.
4. Bookmarks Section.

2.2.3.2 Hardware Interfaces

No direct hardware interaction apart from client devices (desktop/laptop/mobile).

2.2.3.3 Software Interfaces

Database: PostgreSQL with vector similarity search.

Frontend: Next.js, Axios

APIs: RESTful API endpoints built using Django and DRF for article retrieval, recommendations, and user activity logging.

Libraries: Hugging Face Transformers (summarization, embeddings), Celery + Redis (background tasks).

2.2.4 Other Non-functional Requirements

2.2.4.1 Performance Requirements

1. The system must support at least 1,000 concurrent users.
2. Article scraping latency should not exceed 15 minutes for new content.
3. Clustering, Summarization, Sentiment Analysis & Political Bias detection processing time should not exceed 15 minutes for new content.
4. The recommendation engine should respond within 500 ms for user queries.

2.2.4.2 Safety Requirements

1. Backup and recovery mechanisms for database failures.
2. Other components of the system should continue working if one component become unavailable.

2.2.4.3 Security Requirements

1. Role-based authentication and authorization.
2. HTTPS communication across all services.
3. Protection against scraping detection and API rate-limits by implementing throttling strategies.

2.3 Cost Analysis

Cost Analysis Assumptions

The following cost analysis provides a detailed estimate of the monthly operational expenses required to run the NewsInsights platform. The figures presented in the subsequent tables are based on a set of foundational assumptions regarding the hosting environment, usage scale, and third-party service models. It is important to consider these assumptions as they provide the context for the financial projections.

1. Hosting Environment:
   The costs are calculated based on a Virtual Private Server (VPS) or a comparable Infrastructure-as-a-Service (IaaS) cloud model from major providers (e.g., AWS, Google Cloud, DigitalOcean). This assumes we are renting virtualized hardware rather than purchasing physical servers.

2. Pricing and Currency:
   All financial figures are estimates presented in Indian Rupees (INR) and are based on current market rates as of Q3 2025. Actual costs may vary depending on the chosen provider and any future price fluctuations.

3. User Load:
   The infrastructure is sized to support an initial target of approximately 5,000 to 10,000 Daily Active Users (DAU). It is further assumed that an average active user reads 6-8 clustered stories per day. This translates to an estimated 40,000 to 60,000 article cluster views per day, culminating in approximately 1.2 to 1.8 million views per month. The costs for the database and application server are sized to handle this level of read traffic and the associated user event logging.

4. AI Model Deployment:
   The costs for AI models are based on a daily ingestion rate of approximately 1,500 to 2,500 new articles. This volume translates directly to the processing load on our AI services, requiring roughly 2,000 sentiment analysis and political bias checks per day. Furthermore, this ingestion rate is expected to create 300 to 500 new story clusters daily, each of which must be processed by the GPU-intensive summarization model. The high costs associated with the AI inference endpoints are a direct function of this daily content processing volume.

5. Scope of Cost:
   This analysis is strictly limited to recurring monthly operational costs. It excludes one-time setup fees, domain registration (which is annualized), software licensing (as the stack is primarily open-source), and any human resource costs related to development, management, or maintenance.

The operational cost for the NewsInsights platform's infrastructure is based on a modern, modular architecture designed for scalability and performance. As detailed in Table 3: Monthly Infrastructure Cost Breakdown, the total estimated monthly expense ranges from ₹19,600 to ₹27,150. This budget covers all essential components required to host the application, manage data, and serve a significant user base, ensuring a responsive and reliable user experience from launch.The primary Application Server, projected to cost between ₹5,000 and ₹7,000, is designated to run the Django backend and the Celery workers responsible for data processing. A significant portion of the budget is allocated to a more powerful Database Server, estimated at ₹10,000 to ₹13,000, which is necessary to handle the memory-intensive vector search operations required by PostgreSQL with the pgvector extension. This setup is complemented by a dedicated Redis Server (₹1,500 – ₹2,500) for efficient caching and task queuing, along with essential allocations for block storage and data transfer, ensuring a robust and responsive system.

The most significant cost driver within this budget is the Database Server, a deliberate investment reflecting its critical role in powering the platform's core AI recommendation feature. By allocating substantial resources to the database, we ensure that the computationally expensive similarity searches remain fast, providing a seamless and personalized user experience. This balanced allocation ensures that while the overall cost is managed, performance is not compromised, establishing a scalable foundation capable of handling the initial demands of data ingestion and user traffic.

Table 3: Monthly Infrastructure Cost Breakdown

Component	**Minimum Requirement**	**Estimated Monthly Cost (INR)**	**Justification**
Application Server	8 vCPU, 16 GB RAM	₹5,000 – ₹7,000	To run the Django application, DRF APIs, and Celery workers for scraping and processing. The custom political bias model would also run here.
Database Server	8 vCPU, 32 GB RAM	₹10,000 – ₹13,000	For PostgreSQL with the pgvector extension. Vector similarity searches are memory-intensive, requiring a powerful instance for good performance.
Redis Server	2 vCPU, 8 GB RAM	₹1,500 – ₹2,500	To act as the Celery message broker and for application-level caching.
Block Storage	~200 GB	₹1,500 – ₹2,000	For storing news articles, user data, system logs, and database backups.
Data Transfer	~2-3 TB/month	₹1,500 – ₹2,500	To account for data from web scraping and traffic from users accessing the platform.
Domain & SSL	N/A	₹100 – ₹150	Standard annual cost of a domain name and SSL certificate, broken down monthly.
Subtotal		**₹19,600 – ₹27,150**	

Beyond the core infrastructure, the platform's intelligence layer is powered by a suite of specialized AI and Machine Learning models, whose costs are outlined in the Table 4: AI/ML Models Cost Breakdown. A hybrid hosting strategy has been adopted to balance performance and expense. The lightweight models for embedding generation and the custom political bias detection are designed to be self-hosted, running directly on the main Application Server, thus their costs are absorbed within the infrastructure budget. In contrast, the more demanding services are deployed on dedicated inference endpoints. The Sentiment Analysis model operates on a modest CPU instance, costing an estimated ₹4,000 to ₹5,000 per month. The most significant operational expense is the Summarization service, which requires a dedicated GPU-powered instance to handle the Mistral-3B model effectively, incurring a substantial cost of approximately ₹30,000 to ₹35,000 monthly.

Table 4: AI/ML Models Cost Breakdown

Model / Service	**Hosting Strategy**	**Estimated Monthly Cost (INR)**	**Justification**
Embeddings (all-MiniLM-L6-v2)	Self-hosted on App Server	Included in Infrastructure	This model is lightweight and can efficiently run on a CPU within a Celery task.
Political Bias Detection	Self-hosted on App Server	Included in Infrastructure	The custom-trained model is assumed to be optimized to run on the main application server's CPUs.
Sentiment Analysis (cardiffnlp/twitter-roberta-base-sentiment)	Hugging Face Inference Endpoint (CPU Instance)	₹4,000 – ₹5,000	This model runs well on a small, dedicated CPU instance for reliable performance without burdening the main server.
Summarization (Mistral-3B)	Hugging Face Inference Endpoint (GPU Instance)	₹30,000 – ₹35,000	This is a large model that requires a GPU (like an NVIDIA T4) for acceptable inference speed. Running it 24/7 on a dedicated endpoint is a significant cost.
Subtotal (AI/ML)		**₹34,000 – ₹40,000**	

Conclusion

In conclusion, the total estimated monthly operational cost for the NewsInsights platform, combining both infrastructure and specialized AI services, falls within the range of ₹54,000 to ₹67,000. It is crucial to note that the AI/ML services, driven almost entirely by the GPU requirement for the summarization model, constitute the majority of this expenditure. This financial distribution underscores a strategic decision to invest heavily in the advanced, user-facing features that define the platform. While the infrastructure provides a robust and scalable foundation, it is the significant investment in AI that delivers the core value proposition of intelligent, concise, and insightful news analysis, justifying the cost as essential to the project's success.

2.4 Risk Analysis

1. Scalability Risks:
   The system may face performance degradation if user traffic exceeds expected capacity especially during peak news cycles or major events.

2. Dependency Risks
   Reliance on third-party APIs (Hugging Face Inference API, RSS feeds, external NLP services) may cause service disruption or unexpected costs if access is limited, changed, or revoked.

3. Data Quality Risks:
   Since the platform depends on external news sources, misinformation, biased reporting, or incomplete data at the source level can propagate into the system, reducing credibility.

4. Model Accuracy Risks:
   The political bias detection, sentiment analysis, summarization, and recommendation models may produce inaccurate or misleading outputs, affecting trust and user experience.

5. Security & Privacy Risks:
   Handling user interaction data (clicks, reading patterns,preferences) introduces risks of unauthorized access, data leakage, or misuse if strong security practices are not enforced.

6. Cost Overrun Risks:
   Usage-based pricing of inference APIs may lead to higher-than-expected operational costs if user adoption scales quickly, especially for sentiment and summarization tasks.

7. User Adoption Risks:
   Initial cold-start problems in recommendation systems for new users or limited engagement with bias-detection features may reduce the system's perceived value.

Methodology Adopted

3.1 Investigative Techniques

The detection of political bias in news articles is a complex, multi-layered challenge that combines linguistics, political science, communication theory, and computational modeling. To address this effectively, investigative techniques must be technically robust and conceptually aligned with the nuances of media bias. Our chosen methods—Transformer-based models, contextual embeddings, custom similarity-driven clustering, and a hybrid pipeline—are justified both by research trajectories in the field and by the unique requirements of analyzing bias in the Indian political context.

Justification for Transformer Models (RoBERTa, DeBERTa)

Traditional approaches such as lexicon-based systems and SVMs struggle to capture the subtlety of framing, omission, and epistemological biases. Transformer models like RoBERTa and DeBERTa excel because their self-attention mechanisms capture long-range dependencies and nuanced contextual signals.

1. Evidence from Literature: Prior studies have demonstrated that BERT-family models achieve state-of-the-art performance on bias detection datasets such as BASIL, particularly in sentence-level framing analysis.
2. Why We Use Them: RoBERTa and DeBERTa are pre-trained on massive, diverse corpora, ensuring strong generalization. DeBERTa's disentangled attention further refines positional bias handling, a subtle but relevant aspect of sentence framing in political reporting.

Contextual Embeddings via Sentence-BERT

Bias often emerges not from single words but from semantic framing at the sentence or article level. Sentence-BERT embeddings allow us to represent news articles in a dense semantic space that supports cross-outlet comparisons.

1. Why This Matters: Embeddings enable comparative framing analysis—for example, contrasting coverage of a protest described as "defending democracy" versus "causing disruption."
2. Justification: This approach aligns with communication theories such as Framing Theory and Agenda-Setting Theory, which emphasize that media bias is often expressed through issue framing rather than factual distortion.

Event Clustering via Custom Similarity-Based Pipeline

A major innovation in our work is clustering articles into real-world events before classification, using a custom pipeline rather than traditional clustering algorithms like DBSCAN or k-means.

1. Gap in Literature: Most prior research classifies articles independently, missing the comparative context bias detection requires.

2. Our Approach: Articles are clustered based on:
    a. Sentence-BERT embeddings, normalized for consistency.
    b. Temporal filtering (72-hour window) to ensure event coherence.
    c. Deduplication via content hashing to avoid repeated articles skewing clusters.
    d. Category boosting, which slightly increases similarity scores for articles of the same topic/category.
    e. Dynamic centroid updates as new articles join clusters.

    Justification: This pipeline ensures clusters reflect real-world event groupings, enabling direct comparisons across outlets and capturing both framing bias and omission bias, which are otherwise extremely difficult to detect.

Hybrid Unsupervised + Supervised Pipeline

We integrate unsupervised event clustering with supervised bias classification:
Why Hybrid? News scraped from the web is noisy and unlabeled. Clustering imposes structure, while supervised models (RoBERTa/DeBERTa) assign bias labels.
Justification: This two-stage process mirrors real-world conditions—chaotic news streams first organized into events, then labeled contextually. This increases robustness and ensures classification occurs in meaningful, event-aware groupings.

Suitability for the Indian Political Context

Existing datasets (BASIL, AllSides, MBIB) are US-centric. Our project specifically targets Indian political discourse, requiring techniques that can adapt to unique linguistic and cultural markers of bias.

Why Our Techniques Work Here:

1. Fine-tuned Transformers adapt to India-specific rhetorical patterns.
2. Event clustering allows analysis of how Indian outlets cover the same events (e.g., elections, protests, reforms).
3. Contextual embeddings capture cultural framings such as caste, religion, and region-specific terminology.

Conclusion

Our investigative techniques such as Transformer models, contextual embeddings, custom similarity-based clustering, and a hybrid pipeline all align with the state of the art while addressing gaps in context-awareness, generalization, and cultural applicability. This ensures our system is both academically rigorous and practically impactful for the Indian news landscape.

3.2 Proposed Solution

1. Data Aggregation
   The system begins by aggregating raw news content from multiple online sources. A custom web scraping pipeline continuously extracts articles from ten distinct news channels, ensuring diversity in coverage across categories such as India, World, Business, Technology, Entertainment, Sports, Science, and Health.
   Each article is stored in a structured database model with fields including title, content, publication timestamp, source, and category. Deduplication is enforced through source-based uniqueness constraints on URLs.
   This aggregation layer ensures a centralized repository of heterogeneous news data, serving as the input for subsequent clustering and analysis stages.

2. Clustering of News Articles
   To group semantically similar news articles, the system employs the SentenceTransformer model `all-MiniLM-L6-v2`. Each incoming article is preprocessed by combining its title and body content, removing excessive whitespace, and irrelevant characters. The resulting cleaned text is embedded into a 384-dimensional vector space.
   Deduplication is performed using hash-based checks on the article titles, preventing near-duplicate entries. For clustering, embeddings are normalized and compared with existing cluster centroids using cosine similarity. A similarity threshold of 0.75 is applied, with an additional category-based boost of 0.1 when articles share the same category label. To ensure temporal relevance, only clusters formed within the last 30 days are considered for assignment.

   The clustering process follows two cases:

a. If the similarity score with an existing cluster centroid exceeds the threshold, the article is added to that cluster. The cluster centroid is then updated as the normalized mean of its member embeddings.
b. If no suitable cluster is found, a new cluster is created, and the article is designated as its first member.

   To maintain efficiency, cluster sizes are bounded between one and fifty articles. Embeddings are persisted in the database to ensure consistent retrieval and incremental updates.

3. Summarization and Sentiment Analysis
   The system integrates transformer-based models for higher-level news understanding. Summarization is performed using the Mistral-3B model, while sentiment classification is handled by the RoBERTa model (`cardiffnlp/twitter-roberta-base-sentiment`). These models generate concise summaries, sentiment polarity, and confidence scores for each cluster, enhancing interpretability and user experience.

4. Political Bias Detection

   Political bias detection is the central pillar of the proposed system. While existing approaches often rely on classifying single articles in isolation, our design emphasizes event-level comparison across multiple outlets to capture subtler forms of framing and omission bias.

   a. Central pillar of the system → focuses on identifying political bias not only in single articles but through event-level comparison across multiple outlets.
   b. Nature of bias:
      i. Rarely overtly partisan.
      ii. Emerges through story selection, framing, and omission.
      Example: One outlet highlights economic benefits of a policy; another emphasizes public protests.
   c. Event clustering:
      i. Articles grouped into coherent event sets using Sentence-BERT embeddings
      ii. Temporal and categorical similarity applied for robust clustering.
      iii. Ensures all reports of the same incident are analyzed together.
   d. Bias classification::
      i. Within each cluster, transformer-based classifiers (RoBERTa/DeBERTa) are fine-tuned for bias detection.
      ii. Detects bias at both:
         I. Sentence level → loaded words, presuppositions, sentiment polarity.
         II. Article level → framing choices, omission of critical perspectives.
   e. Advantages of integration::
      i. Combines clustering + classification to detect framing and omission bias.
      ii. Captures differences in how outlets report the same event.
      iii. Overcomes limitations of single-document models.
   f. Hybrid design:
      i. Unsupervised stage: Organizes noisy, real-world news into structured clusters.
      ii. Supervised stage: Assigns bias labels to clusters and articles.
      iii. Ensures scalability, robustness, and adaptability to continuous news flows.
   g. Explainability & transparency:
      i. Outputs contextual evidence (highlighted biased sentences + cross-outlet comparisons)
      ii. Aligns with Explainable AI (XAI) principles.
      iii. Aims to make bias detection accurate, interpretable, and educational for users.

5. Personalized News Recommendation
   The system incorporates an AI-powered recommendation engine that leverages user interaction data and semantic similarity of news clusters. User interactions are tracked, which records events such as clicks, reads, likes, and bookmarks.

   a. User Profile Construction: A user profile embedding is computed by averaging the vector representations of the clusters with which the user has interacted. These embeddings are normalized to ensure robust cosine similarity comparisons.

   b. Recommendation Retrieval: To generate recommendations, the user profile embedding is compared against candidate cluster embeddings stored in the database. The system employs PostgreSQL with the `pgvector` extension, enabling efficient nearest-neighbor search using cosine distance. Clusters that are semantically closest to the user profile but not yet interacted with are selected as recommendations.

   c. Bookmarking and Personalization: Users can bookmark clusters for later access, with bookmarks also contributing to the recommendation process. This supports both short-term preferences (recent reads) and long-term personalization (saved content).The recommendation preferences (recent reads) and long-term personalization (saved content). The recommendation engine ensures that each user receives a personalized, semantically relevant news feed, while maintaining diversity by sourcing articles from multiple perspectives.

3.3 Work Breakdown Structure

This diagram represents the overall workflow of NewsInsights, structured as a hierarchical process. At the top level, it captures inputs (news articles), which then flow into functional components like preprocessing, clustering, and classification. The event clustering stage groups related articles together using Sentence-BERT embeddings, temporal filters, and similarity measures. After clustering, the bias detection stage applies transformer-based classifiers (RoBERTa/DeBERTa) to label clusters/articles with bias categories. Finally, the system generates outputs and insights, providing context-aware comparisons across outlets and interpretable explanations of bias patterns.

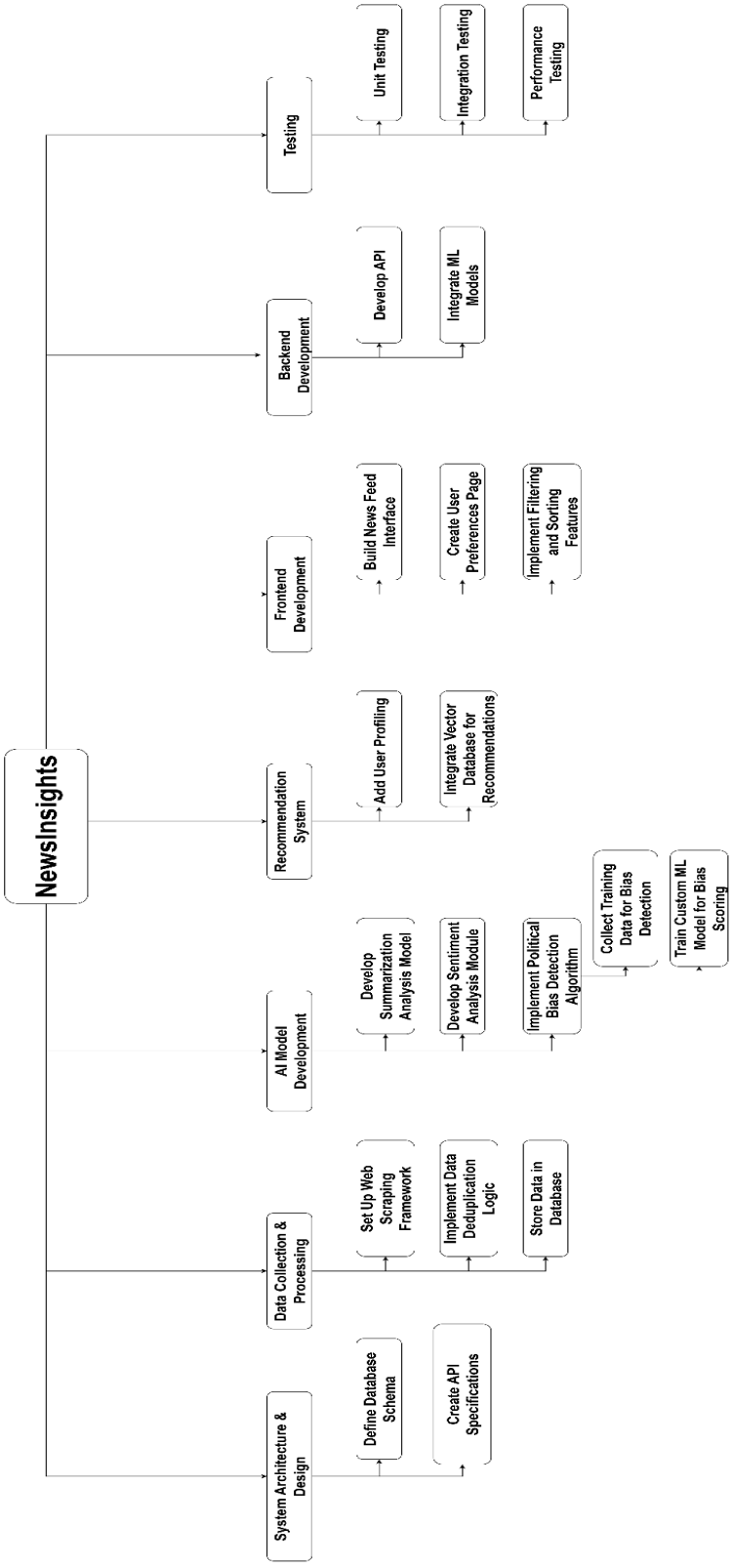


Figure 1: Work Breakdown Structure

The development of the NewsInsights platform is organized into seven primary work packages as shown in Figure 1. This structure provides a clear roadmap for development, covering everything from initial design to final testing and deployment.

1. System Architecture & Design
   This foundational phase establishes the technical blueprint. It involves designing the PostgreSQL database schema to structure all platform data and creating detailed API specifications to define the communication protocol between the frontend and backend.

2. Data Collection & Processing
   This work package focuses on building the data ingestion pipeline. It includes setting up parallel web scrapers to fetch news from multiple sources, implementing deduplication logic to ensure data quality, and storing processed articles and their vector embeddings.

3. AI Model Development
   This package covers the creation of the platform's core intelligence. The main deliverables are three functional AI modules: a transformer-based model for summarization, a sentiment analysis classifier, and a custom-trained model to detect political bias in articles.

4. Recommendation System
   This component is dedicated to personalizing the user experience. It involves tracking user interactions and leveraging this data with article embeddings to build a recommendation engine that suggests relevant content.

5. Frontend Development
   This package involves building the entire user-facing application with React. The work covers creating all necessary pages (homepage, authentication, bookmarks) and implementing key features like filtering and sorting.

6. Backend Development
   This work package focuses on creating the server-side logic using Django. The primary tasks are to build the REST APIs that handle user authentication, news feeds, and recommendations, and to integrate the AI models into the main application.

7. Testing
   This final package ensures platform reliability and performance. It employs a multi-layered strategy, including unit tests for individual functions, integration tests for module interactions, and performance tests to validate system speed and scalability.

3.4 Tools and Technology

1. Backend Development
    a. Python: The primary language for backend and AI development due to its extensive
    b. libraries.
    c. Django: A high-level Python framework for building the core backend application logic.
    d. Django REST Framework (DRF): A toolkit for rapidly developing RESTful APIs to serve data from the backend.

2. Frontend Development
    a. Next.js: A JavaScript library used to build the dynamic and interactive single-page user interface.
    b. HTML5, CSS3, JavaScript: Fundamental web technologies for creating the client-side application.

3. Database
    a. PostgreSQL: A robust relational database for storing all application data, including articles and user profiles. pgvector is used to store high-dimensional vectors directly in PostgreSQL tables.

4. AI & ML
    a. Hugging Face Transformers: A library providing pre-trained NLP models for text summarization and sentiment analysis.
    b. Scikit-learn + PyTorch: Machine learning libraries for training the custom political bias detection model.

5. Asynchronous Task Processing
    a. Celery: A distributed task queue for running background processes like web scraping and AI inference.
    b. Redis: An in-memory data store used as a message broker for Celery and for application caching.

6. Infrastructure & Deployment
    a. Docker: A containerization platform to ensure consistent deployment and development environments.
    b. Git: A version control system for managing the codebase and facilitating team collaboration.

4.1 System Architecture

The diagram in Figure 2: System Architecture illustrates the system architecture of NewsInsights, showing the flow of news data from collection to end-user delivery. News articles are first gathered from multiple sources (e.g., Indian Express, Hindustan Times, Zee News) through the data stream service. These are stored in the news database, then processed through the AI representation module, which creates embeddings for clustering and bias analysis. User-specific preferences are also stored separately in a user profile database, enabling personalization. Analytical services (bias detection, sentiment analysis, summarization) operate on the processed data and feed results into the backend API, which finally powers the website interface, providing users with context-rich and bias-aware news insights.

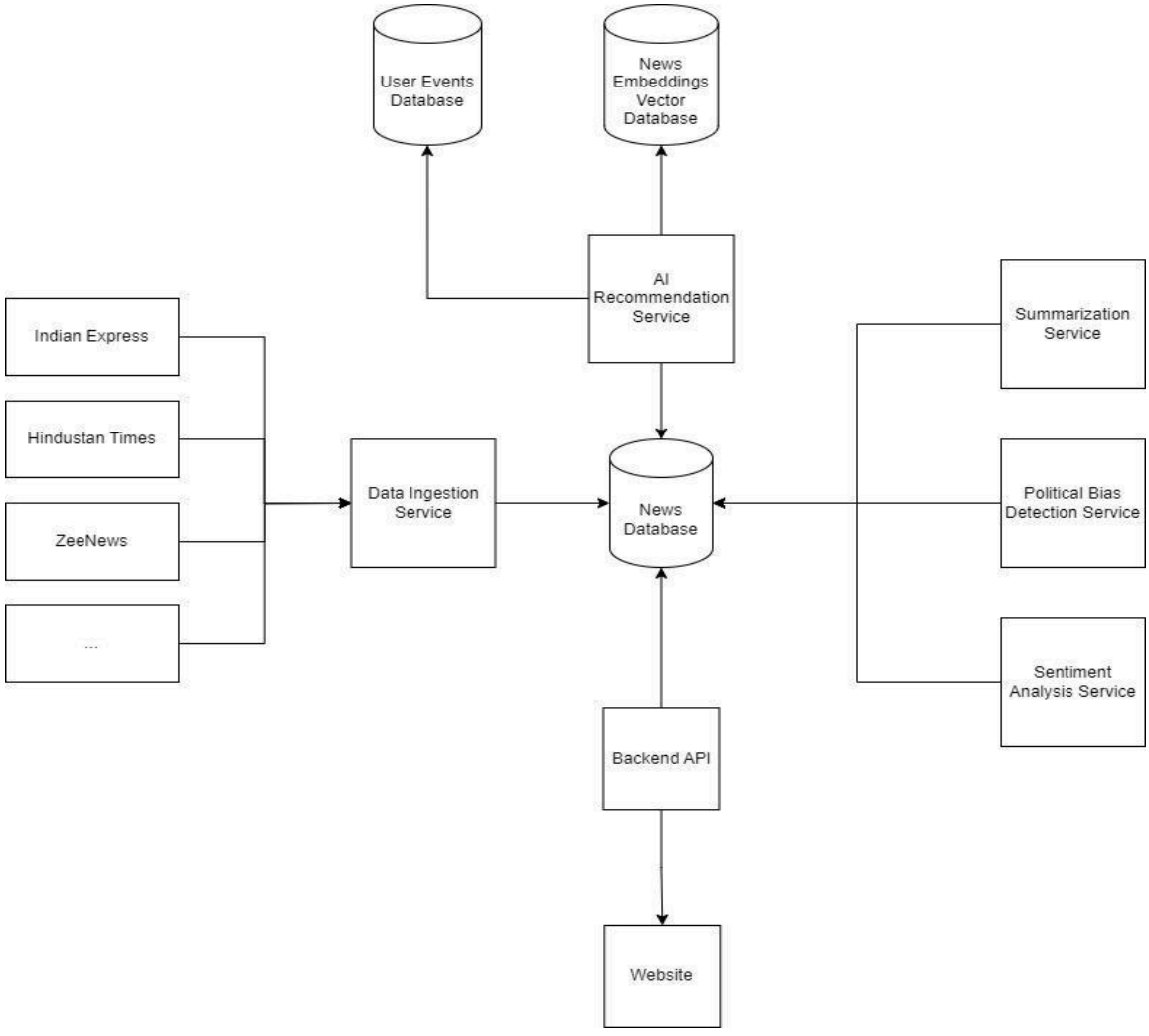


Figure 2: System Architecture

The diagram in Figure 2: System Architecture, illustrates the data flow and interactions between the various components of the NewsInsights platform. It can be broadly divided into several key services and data stores.

1. Data Ingestion Service
   The Data Ingestion Service is the frontline of the NewsInsights platform, responsible not only for collection but also for the initial processing and organization of all incoming news. It employs a fleet of web scrapers to autonomously gather articles from diverse sources. Crucially, as data flows in, this service performs deduplication to identify and discard redundant articles, ensuring data integrity. Immediately following this, it orchestrates the semantic clustering by converting articles into vector embeddings and grouping them based on conceptual similarity. This dual process of cleaning and organizing ensures that all downstream services receive a reliable, well-structured, and non-redundant stream of information, forming the foundation for all subsequent analysis and recommendation tasks.

2. News Database
   At the core of the architecture lies the News Database, which functions as the central repository and the single source of truth for all article-related information. This database is designed to store not only the raw text and metadata collected by the ingestion service but also the enriched data generated by the AI analysis modules. It holds the generated summaries, sentiment scores, and political bias classifications, linking them directly to their corresponding articles. Its robust design ensures that data is efficiently stored and can be quickly retrieved by the Backend API to serve content to the user or by other services for further processing.

3. AI Analysis Services
   The AI Analysis Services represent the intelligence layer of the platform, where raw news content is transformed into insightful, machine-readable data. This suite of services works in parallel to process articles from the News Database, with each module performing a specialized task. The Summarization Service uses advanced models to distill long articles into concise summaries, the Political Bias Detection Service employs a custom-trained classifier to categorize content, and the Sentiment Analysis Service evaluates the tonal leaning of the text. The outputs from these services provide the valuable, multi-faceted analytical layers that are a core feature of the NewsInsights experience.

4. News Embeddings Vector Database
   The News Embeddings Vector Database is a specialized, high-performance data store designed specifically to handle the vector representations of news articles. After an article is processed, its textual content is converted into a numerical vector that captures its semantic meaning. This database is optimized for extremely fast similarity searches, allowing the system to find conceptually related articles in milliseconds. This capability is the technical backbone for both the news clustering feature and the semantically aware recommendation engine.

5. AI Recommendation Service
   The AI Recommendation Service is the engine of personalization, dedicated to creating a unique and relevant news feed for every user. It operates by synergizing two key data sources: the semantic embeddings from the Vector Database and the interaction logs from the User Events Database. By creating a dynamic profile of a user's interests based on their reading history, the service can intelligently query the vector database to find new, unread articles that are semantically similar to what the user has previously engaged with. This ensures that recommendations are not just topically related but are genuinely aligned with the user's nuanced interests.

6. User Events Database
   The User Events Database is a critical component for personalization, acting as the system's memory of all user interactions. Every action a user takes on the website—from clicking on an article and liking it to creating a bookmark—is captured and stored as an event in this database. This continuous stream of interaction data provides the raw material needed by the AI Recommendation Service to build and refine user profiles. By understanding user behavior, the platform can adapt its suggestions over time, leading to an increasingly personalized and engaging experience.

7. Backend API
   The Backend API is responsible for managing the flow of data and communication between the frontend and all the backend services. It exposes a set of well-defined endpoints that the website can call to perform actions like authentication, fetching news articles, retrieving user-specific recommendations, or submitting user interactions. The API handles all the business logic, authenticates users, and orchestrates the necessary calls to the various databases and AI services, ensuring that data is securely and efficiently delivered to the end-user.

8. Website
   The Website is the user-facing application and the primary touchpoint for all interactions with the NewsInsights platform. Built as a modern, responsive web application, it provides an intuitive interface for users to browse news, view the AI-generated analysis, and discover new content through personalized recommendations. It communicates exclusively with the Backend API to request data and send user activity, effectively separating the user interface from the complex backend processing. Its design is focused on delivering a clean, informative, and seamless user experience across all devices.

4.2 Design Level Diagrams

Data Scraping Activity Diagram

The diagram in Figure 3: Data Scraping Activity Diagram, represents the data ingestion pipeline for NewsInsights. The process begins with an automatic trigger that runs every 10 minutes, fetching content from news feeds. If the HTTP request succeeds, the system proceeds to parse the feed/page and normalize essential fields such as title, summary, URL, timestamp, and source. Before insertion, it checks for duplicates in RawNewsFeed; if found, the item is skipped, otherwise, it is inserted into the database. In case of a failed HTTP request, the system logs an error and applies a backoff mechanism to retry gracefully. This ensures the database remains consistent, up-to-date, and free from redundant entries.

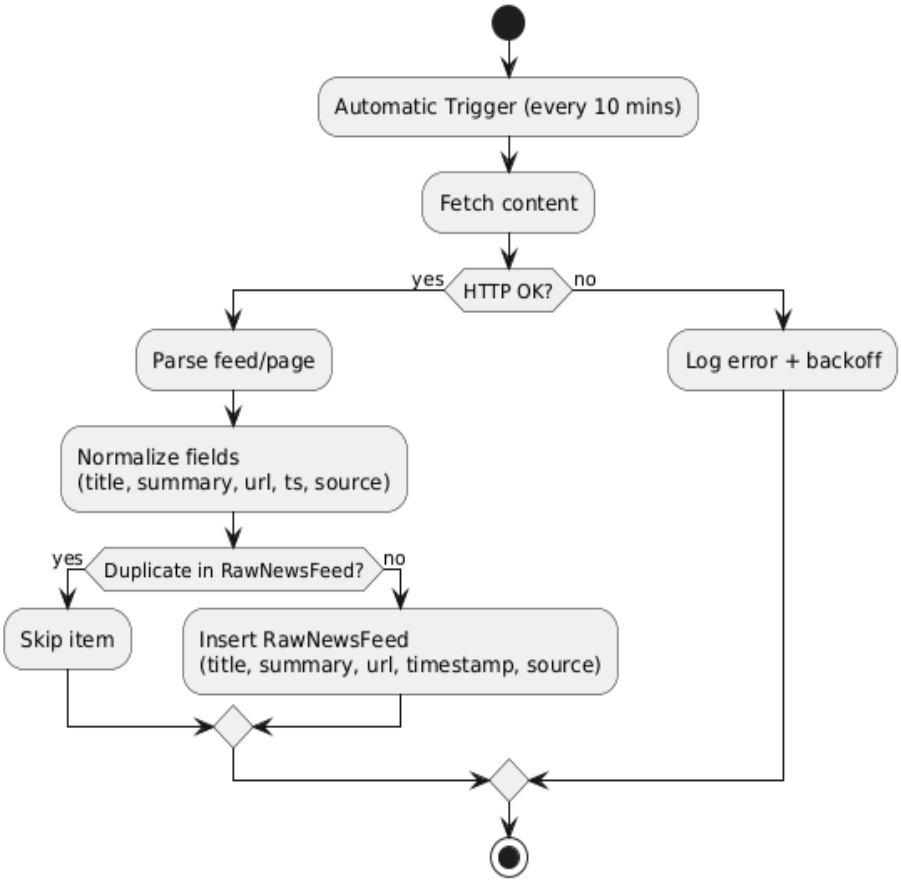


Figure 3: Data Scraping Activity Diagram

Clustering Activity Diagram

The diagram in Figure 4: Clustering Activity Diagram, depicts the event clustering pipeline in NewsInsights. The process begins with a clustering trigger, which loads previously clustered articles alongside new, unclustered ones. If new articles are present, the system applies the clustering model to assign a cluster ID to each article. It then checks the database for matching clusters, if a suitable cluster exists, the article is added; if not, a new cluster is created and the article is assigned accordingly. Each processed article is saved, and the pipeline iterates until all new articles are clustered. This ensures that coverage of the same event (from multiple outlets) is grouped together, enabling meaningful cross-source comparisons and bias detection.

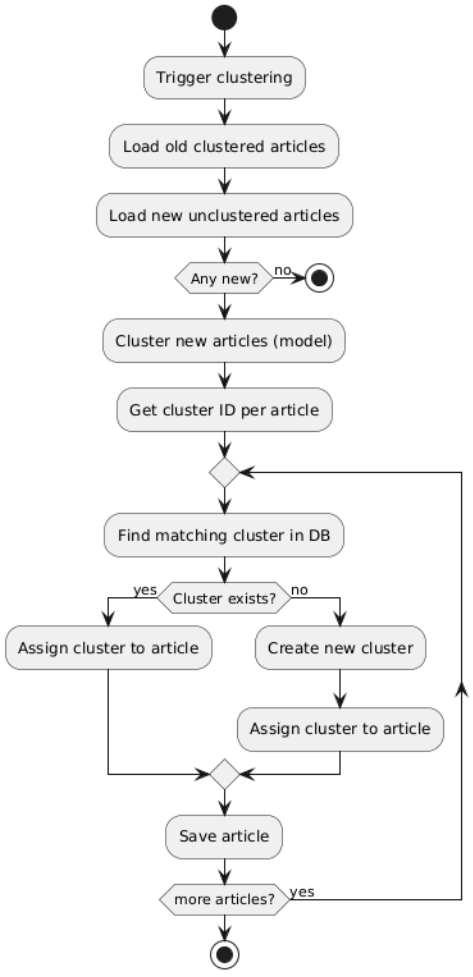


Figure 4: Clustering Activity Diagram

Recommendation System Activity Diagram

The diagram in Figure 5: Recommendation System Activity Diagram outlines the workflow of the personalized news recommendation engine. The process initiates upon an authenticated request, where it first parses parameters defining the response limit and the days for content recency. It then queries the user's historical interaction events to compile a comprehensive set of news cluster IDs with which they have previously engaged. This initial phase includes a critical branching condition to handle the cold start problem: if a user has no prior interaction data, a profile cannot be generated, and the system immediately returns an empty recommendation set, terminating the workflow. For users with a history, this collection of interacted cluster IDs serves as the foundational data for constructing a personalized interest model.

The core of the recommendation logic lies in generating a user profile vector, a high-dimensional numerical representation of the user's latent interests. To achieve this, the system retrieves the pre-computed vector embeddings for all news clusters in the user's interaction history. It then computes the mean of these embedding vectors to create a single, averaged vector. This vector is subsequently subjected to L2 normalization to ensure its magnitude does not skew similarity calculations. The resulting normalized vector acts as a sophisticated digital fingerprint of the user's content preferences, effectively modeling their position within the multi-dimensional semantic space of the news corpus.

With the user profile vector computed, the final phase involves candidate retrieval, ranking, and filtering. The vector is used to perform a similarity search against the entire corpus of news cluster embeddings, explicitly excluding any clusters the user has already seen to ensure novelty in the recommendations. The ranking is executed using cosine similarity, which identifies the candidate clusters that are most semantically aligned with the user's profile. This ranked list undergoes a crucial post-processing step: a timeliness filter is applied to discard any cluster that does not contain articles published after a pre-calculated cutoff date. Finally, the validated, relevant, and timely clusters are serialized into the required JSON format and returned to the user as their personalized feed.

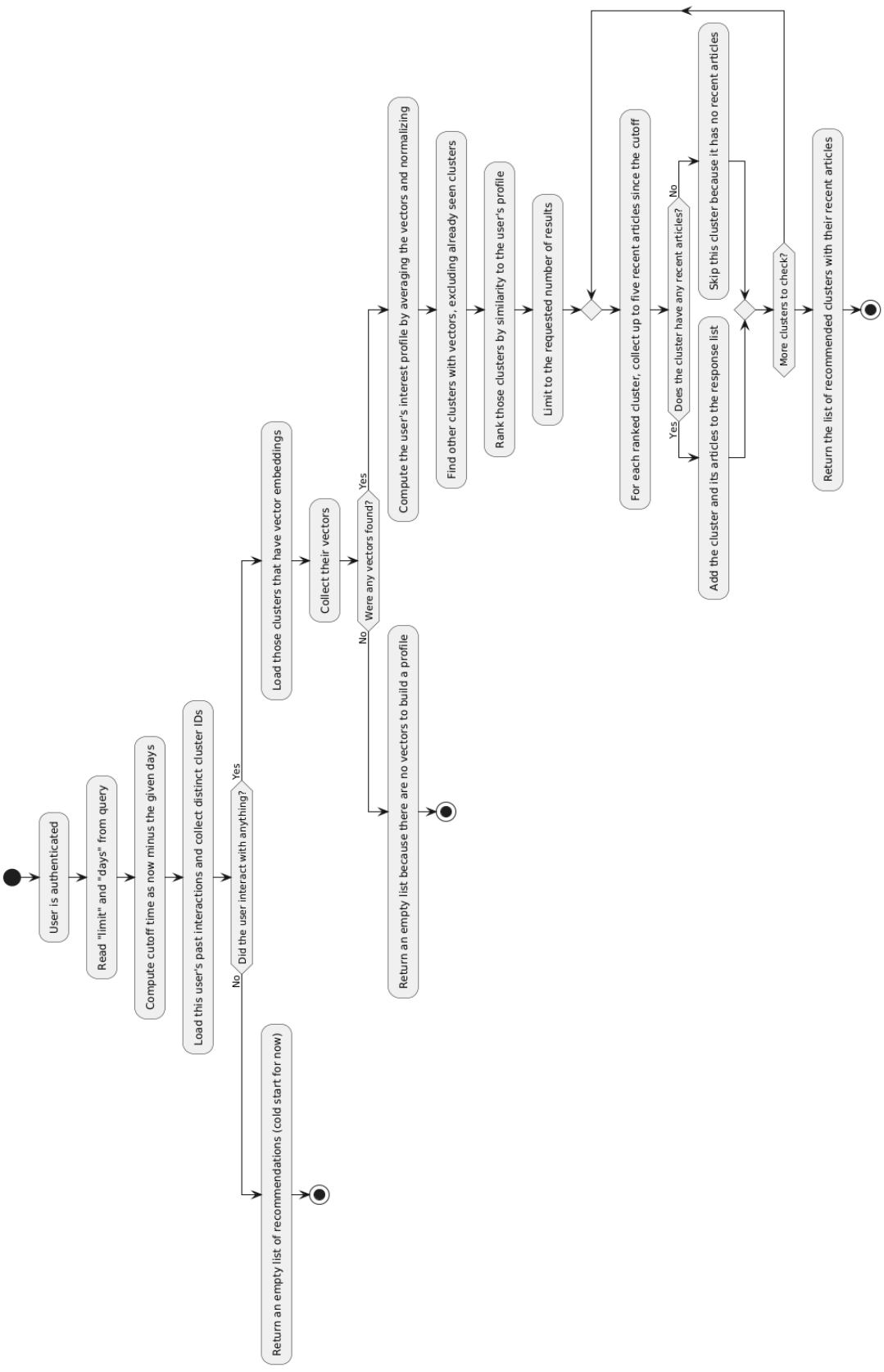


Figure 5: Recommendation System Activity Diagram

ER Diagram

The diagram in Figure 6: ER Diagram illustrates the Entity–Relationship (ER) model for the NewsInsights system. At its core, the schema connects users, events, and clustered news data.

1. The User entity stores account information (username, password, email) and is linked to multiple bookmarks and user events.
2. Bookmark allows users to save clusters for later, recording the cluster ID and creation timestamp.
3. UserEvent captures interactions (e.g., clicks, views) tied to a user and a specific cluster, with event type and timestamp for analytics.
4. RawNewsFeed stores ingested articles with metadata (title, content, source, category, sentiment, embedding vectors) and links each article to its corresponding cluster.
5. Cluster represents groups of related articles, storing summary, sentiment, total views, and embedding vectors for semantic similarity.

Together, this ER model ensures that articles are systematically organized into clusters, while also enabling personalization through user-specific events and bookmarks.

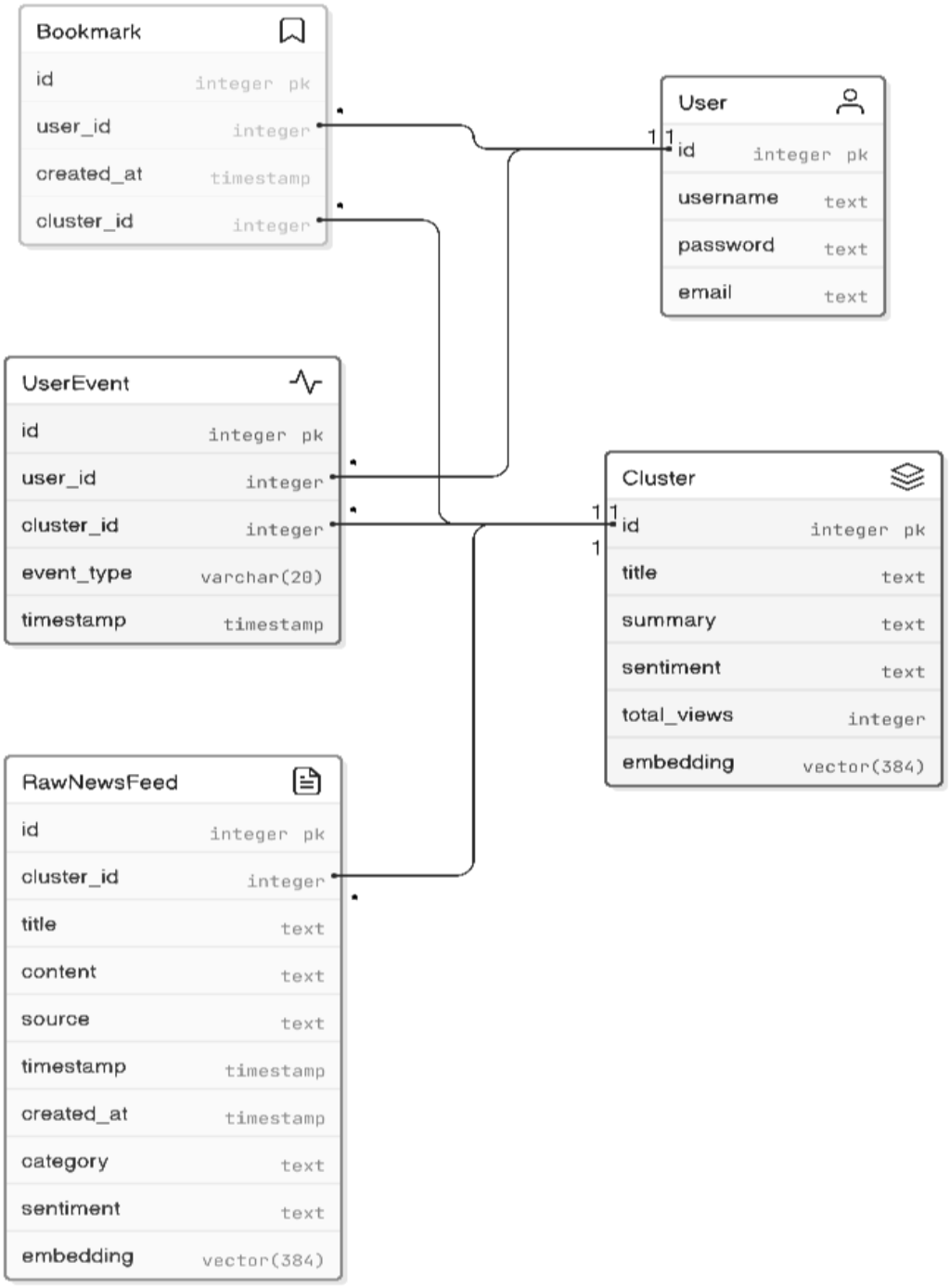


Figure 6: ER Diagram

System State Diagram

The diagram in Figure 7: System State Diagram provides a comprehensive overview of the NewsInsights platform's operational flows, segmented into four primary domains: Account Management, System Processing, Content Exploration, and Personalization. The Account Management section outlines the user authentication lifecycle, beginning with an unregistered user who can register an account. Once registered, the user enters a "Start Not Authenticated" state, from which they can log in. A successful authentication initiates an active session, which can be refreshed to maintain login status or terminated via logout. This section also includes auxiliary flows for checking username/email availability and resetting a forgotten password, encapsulating all user identity and access control processes.

The System Processing block details the platform's automated, continuous content pipeline, which operates independently of user sessions. This lifecycle begins with the ingestion phase, where articles are scraped from various online news sources. These articles are then aggregated, and AI-driven processes generate concise summaries and analyze sentiment. Following this, related articles are grouped into coherent Clusters. In the final stages, both the individual articles and the newly formed clusters are converted into numerical vector embeddings, and the content is filtered to ensure only recent and relevant information is presented to the user, making it ready for discovery and recommendation.

The Content Exploration and Bookmarking sections describe the primary ways an authenticated user interacts with the platform's content. The core loop involves a user browsing news clusters and selecting one to view its detailed summary and source articles. Parallel to this, the bookmarking feature allows a user to check the bookmark status of any cluster and perform actions such as adding it to their saved list, viewing their collection of saved bookmarks, or removing an item. This flow represents the direct, manual engagement a user has with the aggregated news content, forming the basis for their activity profile.

Finally, the Personalization domain illustrates the platform's intelligent features, which create a tailored user experience. Every time a user views a cluster's details, their engagement is tracked. This tracked data serves as a direct input for the recommendation engine, which processes the user's interaction history to generate and present personalized news recommendations. This creates a critical feedback loop where a user's browsing and reading habits actively refine the content they are shown in the future, transforming the platform from a static aggregator into a dynamic, responsive news discovery tool.

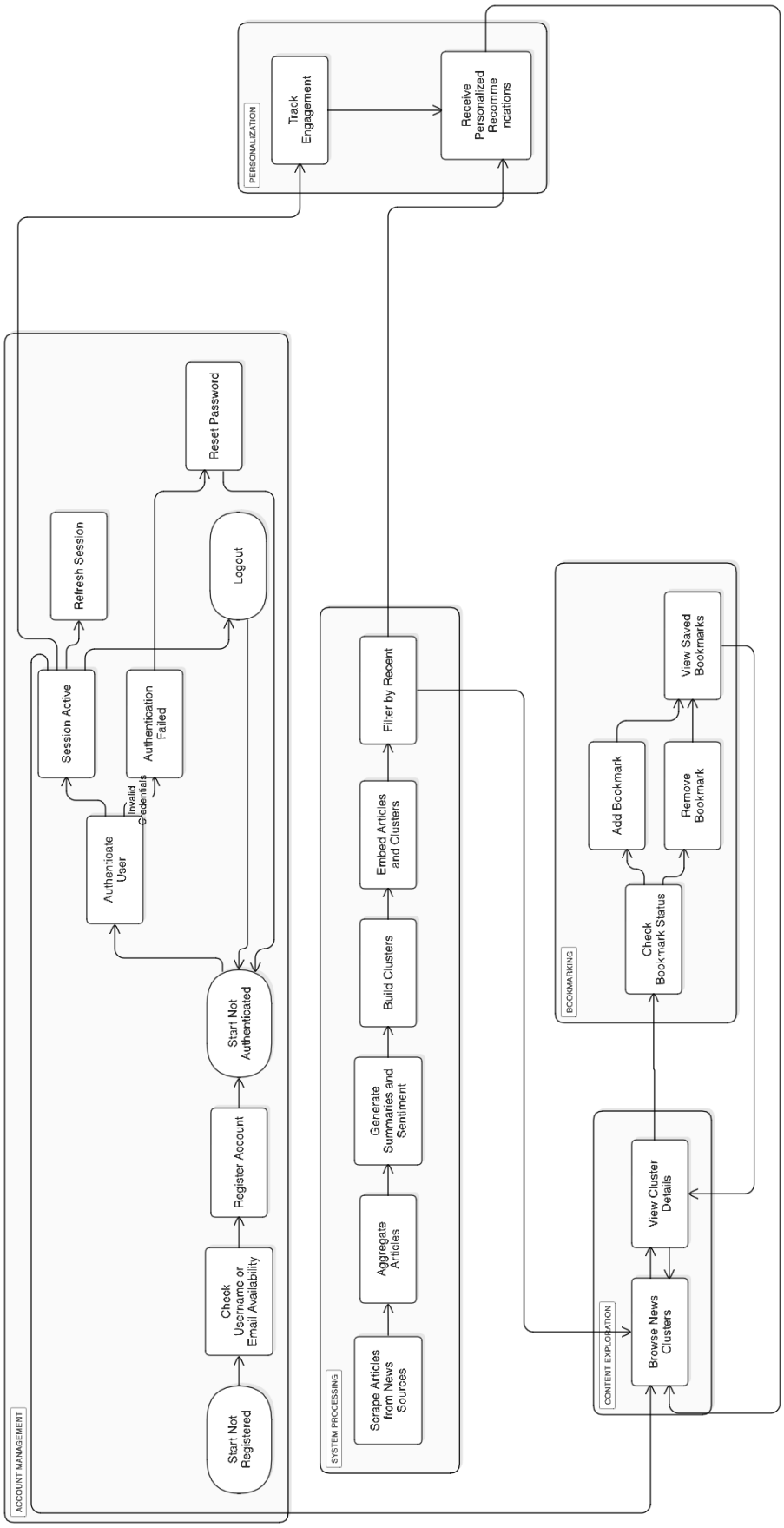


Figure 7: System State Diagram

4.2 User Interface Diagrams

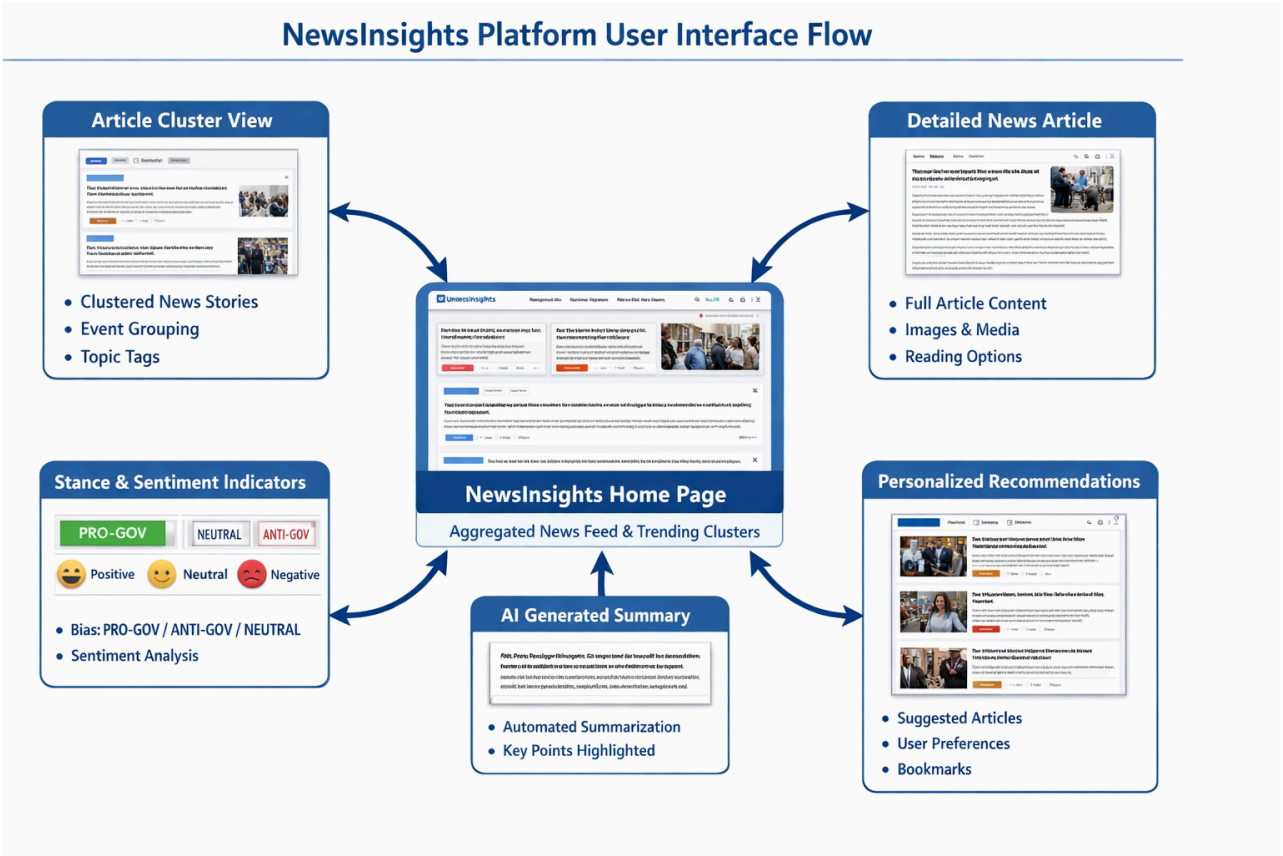


Figure 8: User Interface Diagram

5.1 Experimental Setup

The NewsInsights platform was designed for an end-to-end functionality test of the system's capabilities, such as composite news aggregation, clustering, political bias detection, summarisation, sentiment analysis and recommendations. The end product of the project was a full-stack web application with distinct layers of back-end, AI processing and front-end services.

The back-end layer was created in Django and Django Rest Framework and was responsible for the management of API requests, creation of user interactions, will retrieve articles and communicate with the AI pipeline. The back-end environment included both Celery and Redis to execute the asynchronous background processes of the application. Celery handled the execution of time-consuming web scraping and other tasks generating articles (embeddings), clustering articles, summarising articles and classifying articles into biassed by political viewpoint, and enabled real-time responses to user requests without blocking the requests while going through computationally intensive processes.

The back-end database environment was PostgreSQL and had the package pgvector available for vector similarity searching and clustering recommendations.

In order to integrate and deploy efficiently the AI algorithms responsible for generating summarised articles, sentiment analysis of articles and generating embeddings into the framework and architecture of the project, Hugging Face was used.

5.2 Experimental Analysis

5.2.1 Data

A custom dataset was generated for this project because there wasn’t one publicly available that accurately reflected the unique linguistics, culture and politics involved in Indian news media. The initial articles were collected from well-known Indian media outlets (primarily The Economic Times and The Times of India) since they cover the political, economic, governance and social spheres well and have reliable standards of publication. The data collection process utilised the Scrapy web scraping framework, which made automated, highly scalable and structurally formalised collection of the article content easy. Article metadata (title, date of publication, source, category, full text) was stored in a structured database.

To maintain the quality of the data through preprocessing, the data went through a number of steps: removal of HTML tags, normalising whitespace, removing duplicates and filtering noise. The result was a cleaned, article-level corpus suitable for use in downstream NLP applications. While expert-annotated data sets indicative of political bias are limited for

Indian news, the political bias labelling of the articles utilised large language models (LLMs). The articles were assessed based on agreed guidelines with both Gemini Pro and ChatGPT, in order to classify them as whether they were Left, Right, or Neutral based on the way the articles were framed, their tone, and emphasis rather than factual accuracy.To reduce labeling inconsistencies across LLM-generated outputs, cross-validation was employed. This semi-automated labeling method allowed for the construction of a scalable and contextually aware dataset with improved accuracy, feasibility and reduced ethics violations.

5.2.2 Performance Parameters

The performance of the NewsInsights system was evaluated using a combination of system-level efficiency metrics (data ingestion latency, end-to-end processing time for newly scraped articles, and responsiveness of the recommendation engine) and model-level analytical indicators (io) (confidence scores and probability distributions). The performance at the system level was based on three factors, including the system's speed in collecting data, the amount of time to process newly scraped articles, and how quickly the recommendation engine responded to users when they interact with the system. The submission of asynchronous tasks, which is used to execute all tasks concurrently, allows for fast processing times and does not degrade the user-facing results.

At the model level, the model's ability to determine political stance was measured through the use of confidence values and probabilities produced by the classifier. Each time the model produces predictions for one of the three classes (Neutral, Pro-Government, and Anti-Government), the predictions are based on the probability of each class. If a prediction has a confidence value below a specified threshold, the prediction is marked as being uncertain. This means that the prediction can be interpreted cautiously and will not be forced into a predetermined classification.

The performance of clustering was evaluated qualitatively. The qualitative analysis of clustering involved determining how well the model produced clusters that had a high degree of semantic coherence and how well the model grouped articles describing the same event at different news outlets. The semantic coverage, coherence, and readability were evaluated in the summarization process. Summaries provided by the NewsInsights platform contained coverage of all the necessary contextual information but also reduced the amount of information presented to users, thereby preventing information overload. The combination of all of these factors allowed for the delivery of timely, reliable, and interpretable insights from the NewsInsights platform.

5.3 Working of the Project

5.3.1 Procedural Workflow

The workflow of News Insights consists of many different sections. The first is that articles will get automatically sent to News Insights through automated news ingestion. At the start, News Insights uses automatic news ingestors to gather articles from a number of news sources with the use of Scrapy-based web crawler applications and RSS readers, and it sends each article through several processes, such as preprocessing, removing duplicate articles, extracting article metadata, and storing the article details within a database.

The workflow of News Insights has a second stage where it takes the text of an article that has been preprocessed and deduplicated, and then breaks the article text down into clusters of articles or groups of articles using an algorithm that evaluates the semantic similarity of the different articles to determine the clustering. Each of these clustering algorithms uses a machine learning algorithm called Sentence-BERT to convert the text of the article into dense representations so that the clustering algorithm can evaluate the cosine similarity of the articles and determine if the articles belong in the same cluster. Each cluster has time and category constraints; therefore, the clustered groups represent articles that are reporting on the same event.

Once clustering has been completed, models will be used asynchronously to analyze the different clusters of articles. The political affiliation/stance of an article is identified using a machine learning classifier specifically for classifying three different political opinions (NEUTRAL, PRO_GOV, ANTI_GOV) on news coverage based on the article's title and content. The machine learning algorithm additionally outputs a probability score for the likelihood that an article is classified as one of the three political stances.

Along with political stance detection, the analysis of sentiment, and the utilization of sentiment analysis to identify the emotional or sentiment-based attributes from the clustered articles is what allows News Insights to provide more contextual information about the events being reported.

5.3.2 Algorithmic Approaches Used

1. Semantic Embedding & Representation

The system leverages Sentence-BERT (SBERT) to generate dense semantic embeddings for news articles. Each article's textual content is transformed into a 384-dimensional vector representation that captures contextual meaning beyond simple keyword matching. These embeddings enable:

1. Semantic similarity computation using cosine distance metrics
2. Efficient nearest-neighbor search for related content discovery
3. Cross-lingual semantic alignment for multi-language news sources

2. Clustering Algorithm

Article clustering employs a custom agglomerative approach with the following characteristics:

Table 5 : Clustering Algorithm Table

Component	Implementation
Similarity Metric	Cosine similarity on SBERT embeddings
Threshold	0.75
Temporal Filtering	Articles clustered within a sliding time window to preserve recency(30 days)
Centroid Updates	Dynamic centroid recalculation as new articles join clusters

The algorithm iteratively:

1. Computes pairwise similarities between article embeddings
2. Applies temporal decay weights to prioritize recent content
3. Merges articles exceeding the similarity threshold into clusters
4. Updates cluster centroids incrementally to maintain representativeness

3. Political Stance Detection

Political stance classification utilizes a fine-tuned XLM-RoBERTa transformer trained on Indian political discourse:

1. Model Architecture: XLM-RoBERTa-base with classification head
2. Output Classes: Pro-Government, Anti-Government, Neutral
3. Probability Distribution: Softmax-normalized confidence scores
4. Uncertainty Handling: Predictions below confidence threshold flagged for manual review

4. Sentiment Analysis

Sentiment classification employs a RoBERTa-based sequence classifier:

1. Pre-trained Base: cardiffnlp/twitter-roberta-base-sentiment
2. Classification: Positive, Negative, Neutral polarity detection
3. Aggregation: Cluster-level sentiment derived from weighted article scores

5. Summary Analysis

Cluster summarization uses the Mistral LLM deployed via LM Studio for generative text synthesis. Prompt-engineered abstractive summarization processes concatenated article content to produce coherent multi-sentence summaries capturing key themes and narratives.

6. Recommendation Engine

Personalized recommendations are generated via vector similarity search:

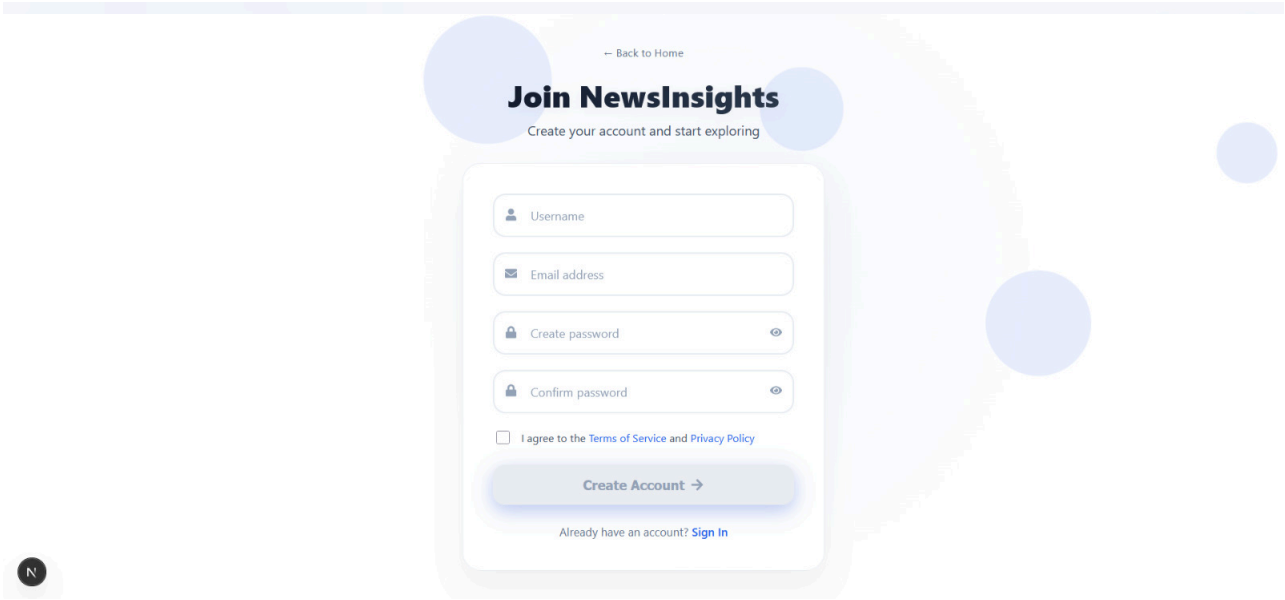
User History Embedding → pgvector Index → K-Nearest Neighbors → Ranked Results

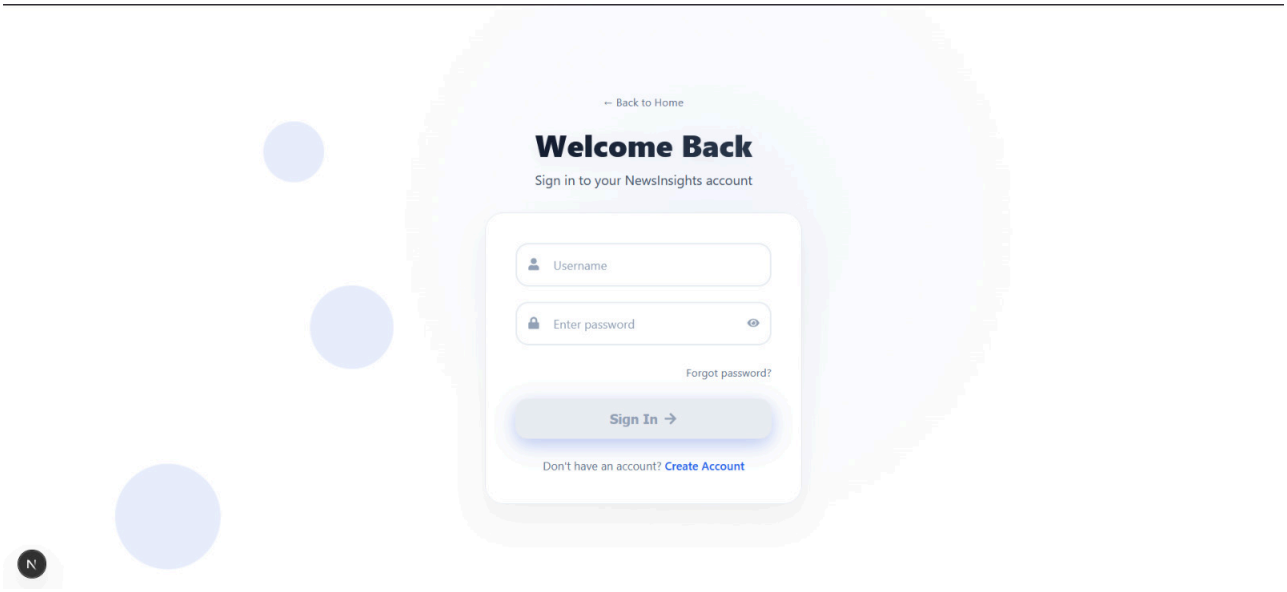
1. Backend: PostgreSQL with pgvector extension for native vector operations
2. Index Type: IVFFlat index for approximate nearest-neighbor search
3. Query Process: User interest profile compared against article embeddings
4. Diversity Injection: Category-aware re-ranking to prevent filter bubbles

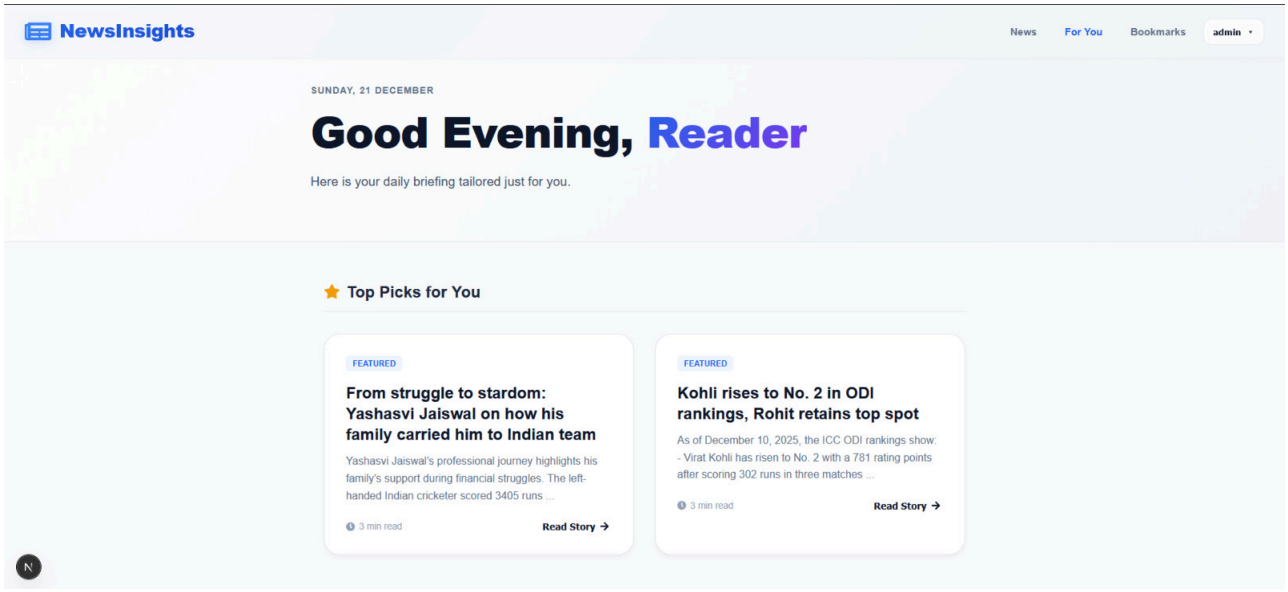
5.3.3 Project Deployment

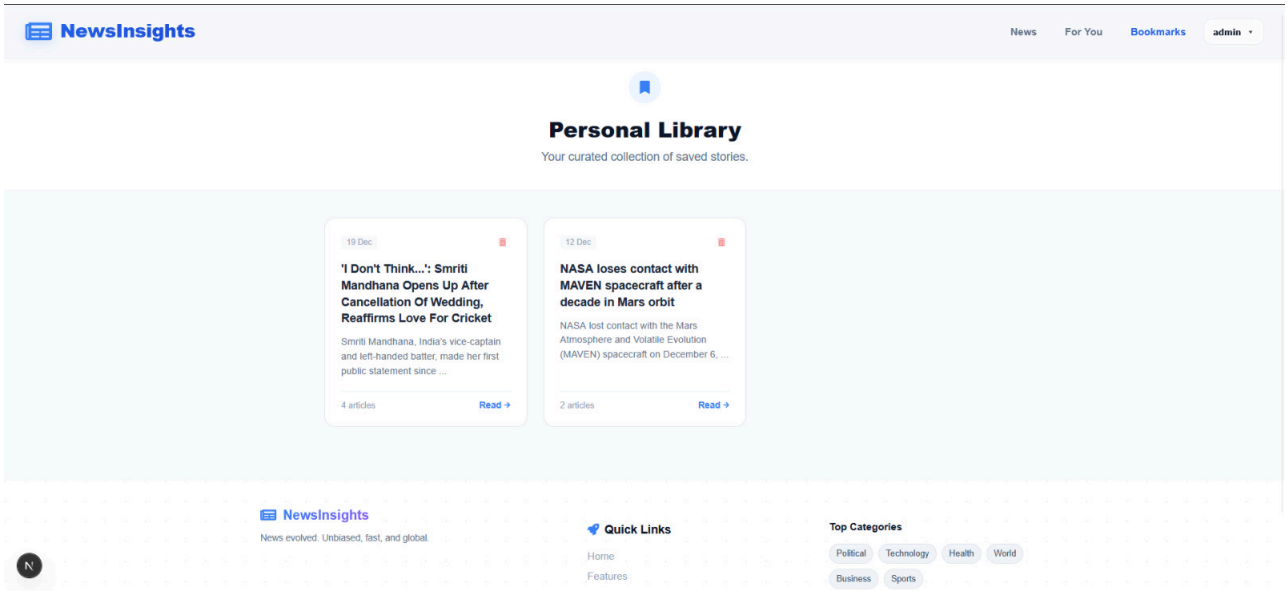
NewsInsights uses the Docker containerization approach to package backend services, background workers, DB components, and cache services. This provides consistent environment setup across all deployments, allows easy scaling of the application component across an environment, and makes the transition from a physical infrastructure to a cloud-based infrastructure less difficult with minimal configuration required.

5.3.4 System Screenshots






NewsInsights
News
For You
Bookmarks
admin
SUNDAY, 21 DECEMBER
Good Evening, Reader
Here is your daily briefing tailored just for you.
Top Picks for You
FEATURED
From struggle to stardom: Yashasvi Jaiswal on how his family carried him to Indian team
Yashasvi Jaiswal's professional journey highlights his family's support during financial struggles. The left-handed Indian cricketer scored 3405 runs ...
3 min read
Read Story →
FEATURED
Kohli rises to No. 2 in ODI rankings, Rohit retains top spot
As of December 10, 2025, the ICC ODI rankings show: - Virat Kohli has risen to No. 2 with a 781 rating points after scoring 302 runs in three matches ...
3 min read
Read Story →


NewsInsights
News
For You
Bookmarks
admin
Personal Library
Your curated collection of saved stories.
19 Dec
'I Don't Think...': Smriti Mandhana Opens Up After Cancellation Of Wedding, Reaffirms Love For Cricket
Smriti Mandhana, India's vice-captain and left-handed batter, made her first public statement since ...
4 articles
Read →
12 Dec
NASA loses contact with MAVEN spacecraft after a decade in Mars orbit
NASA lost contact with the Mars Atmosphere and Volatile Evolution (MAVEN) spacecraft on December 6, ...
2 articles
Read →
NewsInsights
News evolved. Unbiased, fast, and global.
Quick Links
Home
Features
Top Categories
Political
Technology
Health
World
Business
Sports

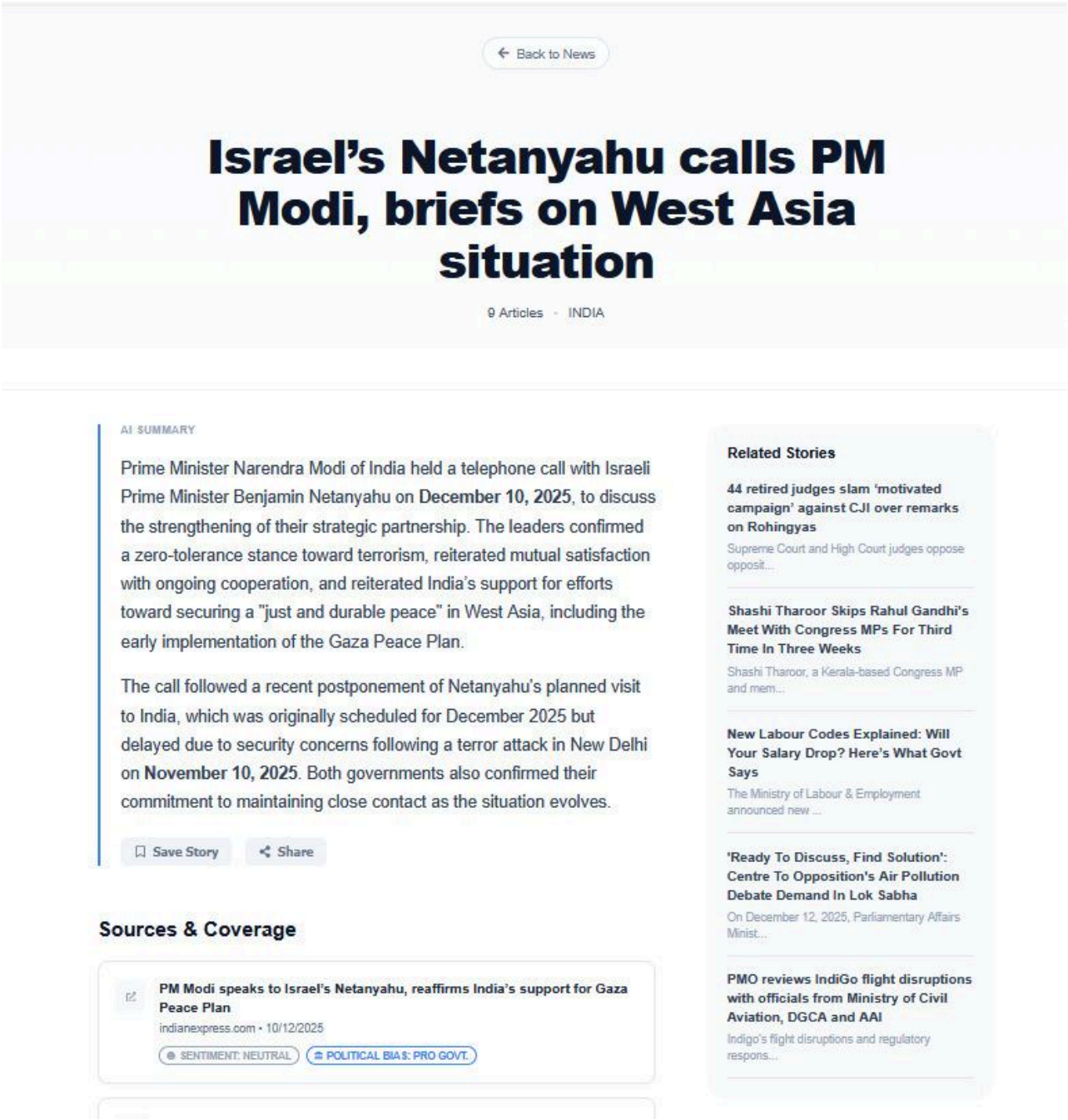


Figure 9 : System Screenshots

5.4 Testing Process

The NewsInsights platform underwent a testing strategy that verified the accuracy, reliability of operations, and interface of the modules that make up the overall system. Emphasis was placed on testing the Data Ingestion and AI analysis components of the platform. Because of how the system

is modularized in the backend, components will be tested at both a module level as well as a Pipeline level to ensure the proper function of the end-to-end system.

5.4.1 Test Plan

Testing Strategy Objectives verify:

1. Web Scrapers (scraping article data from numerous sources) – Reports Support (Clustering, Bias, Sentiment, Summarizing).
2. Execution of Asynchronously Scheduled Jobs – Define how Web Scrapers & Reports Support interact with the data source information in ORCASDL database.

The Test Strategy utilized a hybrid approach of both controlled test cases and production data in order to verify that each of the components mentioned earlier would perform as expected across the entire spectrum of ad-hoc test case scenarios.

.

5.4.2 Features to be tested

The following features were needed:

Web Scrapers

1. Accurate extraction of article title/body text/publication date/source
2. Resiliency to layout changes/missing field
3. Correct handling of duplicate/malformed articles

Clustering Module (clusterHelper.py)

1. Articles in clusters based on semantic similarity as pertaining to the same real-world event
2. Grouping remains stable as additional articles are added
3. Grouping of unrelated articles must be avoided

The software tool 'politicalBiasHelper.py' is for detecting political affiliation (for example, if an article is pro-, anti-, or neutral to the government), assigning confidence scores and uncertainty flags, as well as consistently giving similar articles the same prediction.

The software tool 'sentimentHelper.py' is for performing sentiment analysis (providing the polarity of an article's sentiment, which could be positive, negative, or neutral) and being able to recognize that an article has neutral or mixed tone.

Another software tool, 'summarizeHelper.py', creates accurate and concise summaries of an article while retaining its most relevant content, as well as generating text summaries that do not hallucinate or distort facts.

5.4.3 Test Strategy

There are three distinct components to the testing strategy for each of the NewsInsights modules:

1. Black Box Tests: Test that web parsers will pull out the structured data (title, body and timestamp) from the raw HTML in a manner that does not relate to what happens internally during parsing of the HTML document. For this type of testing we mock requests library where we supply various formats of raw HTML (standard, missing classes and automated boilerplate feed) and check what the output is.
2. Functional Tests: Test that when given a valid input, AI based modules will return outputs that are consistent. For instance using mocks of the underlying AI clients (OpenAI, HuggingFace), we can test how the calls are implemented to AI clients, and verify that logic surrounding AI calls is implemented correctly, including text preparation prior to sending to AI clients, as well as how the results from AI clients will be processed.
3. Boundary Tests: Test that during extreme or edge cases (empty inputs, malformed HTML, extremely small text or any API failure or time out), the system will remain stable..

5.4.4 Test Techniques

The NewsInsights platform was subjected to multiple testing methodologies in order to locate, validate, and assess the reliability and robustness of the application when it operates under production conditions. Since both web-based data sources and AI algorithms are constantly changing and behave probabilistically, the focus on the NewsInsights platform was not only on functional correctness, but also the quality of the final output.

The **black-box testing** methodology was used heavily to validate the output of the web parsers and their corresponding REST APIs. In this manner, the parsers were tested without any knowledge of how they worked internally to ensure that the articles that they extracted were in the correct formats. This was implemented in test_scrapers.py, which tests the HindustanTimes scraper by mocking HTTP responses and verifying that article data is correctly extracted.

Functional tests were applied to all of the AI-based modules (clustering, political stance detection, sentiment analysis, and summarization) to ensure that these modules produced outputs consistent

with their functionality when fed valid information. This provided assurances that the modules would work logically and that they would be correctly integrated into the NewsInsights processing pipeline. These tests are implemented in:

1. test_sentimentHelper.py — Mocks the HuggingFace BERT model to verify sentiment classification logic.
2. test_summarizeHelper.py — Mocks the OpenAI client to verify text preprocessing and summary generation.

By performing **boundary tests**, the NewsInsights developers were able to identify how the application would behave under unusual circumstances, such as processing an empty article, a very short news article, incomplete metadata, an article that was full of "noise," or was improperly formatted. This testing helped ensure that the NewsInsights application would remain stable when faced with unusual data. The boundary test cases include:

1. test_empty_input — Passing None or empty strings to AI helpers.
2. test_garbage_input — Input containing only boilerplate text.
3. test_api_failure — Simulating API connection errors.
4. test_malformed_html — Feeding broken HTML to scrapers.

The qualitative aspects of AI-generated outputs, including coherence, contextual relevance and interpretability, were evaluated through manual validation so as to ensure consistency between human judgement and the AI's output. This was a critical component in determining the accuracy of subjective tasks such as political stance detection and summarisation. Together, these methods allowed for reliable system performance in various and unpredictable data conditions encountered in the real-world.

5.4.5 Test Cases

Scraper Tests (Black-box & Boundary)

Table 6 : Test Case table for Scraper Tests

Test Case	Description	Expected Result
test_clean_article_content	Verify removal of "automated feed" boilerplate.	Boilerplate text is removed; actual content remains.
test_scrape_section_parsing_success	Feed valid HTML with article links to the scraper.	Returns list of dictionaries with correct title, content, category.
test_scrape_empty_section	Feed a valid 200 OK response with no news items.	Returns empty list [].
test_malformed_html	Feed broken/unclosed HTML tags.	Gracefully handles parsing; returns empty or partial list; no crash.

Summarizer Test(Functional & Boundary)

Table 7: Test Case table for Summarizer Tests

Test Case	Description	Expected Result
test_preprocess_text	Verify text cleaning (boilerplate removal, sentence deduping).	Removes "Also Read", "Click here"; keeps sentences > 20 chars.
test_generate_summary_success	Mock successful API response.	Returns summary string matching the mock.
test_empty_input	Pass None or empty string.	Returns None.
test_garbage_input	Pass input containing only boilerplate.	Returns None.

test_api_failure	Mock API raises Exception (e.g., connection error).	Returns None (graceful failure); logs error.

Sentiment Helper Tests (Unit)

Table 8: Test Case table for Sentiment Helper Tests

Test Case	Description	Expected Result
test_query_huggingface_positive	Mock BERT model returning positive logits.	Returns LABEL_2 (Pos) with high score.
test_query_batch	Mock batch processing.	Returns list of results matching input order.

5.4.6 Test Results

Testing confirmed that the NewsInsights platform is a reliable component of its main functions. The web scrape extracts many sets of structured content from different news sources, and the AI Helper generates readable and interpretable output. As well as the predicted political stance, the predicted political stance produced confidence scores, which allows one to interpret the predicted stance with some degree of uncertainty. Asynchronous task execution integrated the entire system and provided for the consistent and uninterrupted operation of the complete process from input to output without interrupting the user experience.

1. Overall System Testing:

Summary:

1. Total Tests Run: 11
2. Passed: 11
3. Failed: 0
4. Errors: 0

```
$ python manage.py test base.tests

Found 11 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).

test_clean_article_content (base.tests.test_scrapers.HindustanTimesScraperTest) ... ok
test_malformed_html (base.tests.test_scrapers.HindustanTimesScraperTest) ... ok
test_scrape_empty_section (base.tests.test_scrapers.HindustanTimesScraperTest) ... ok
test_scrape_section_parsing_success (base.tests.test_scrapers.HindustanTimesScraperTest)
... ok
test_api_failure (base.tests.test_summarizeHelper.SummarizeHelperTest) ... ok
test_empty_input (base.tests.test_summarizeHelper.SummarizeHelperTest) ... ok
test_garbage_input (base.tests.test_summarizeHelper.SummarizeHelperTest) ... ok
test_generate_summary_success (base.tests.test_summarizeHelper.SummarizeHelperTest) ...
ok
test_preprocess_text (base.tests.test_summarizeHelper.SummarizeHelperTest) ... ok
test_query_batch (base.tests.test_sentimentHelper.SentimentHelperTest) ... ok
test_query_huggingface_positive (base.tests.test_sentimentHelper.SentimentHelperTest)
... ok

----------------------------------------------------------------------
Ran 11 tests in 0.040s

OK
Destroying test database for alias 'default'...
```

Figure 10: Output result for unit tests

2. **Political Bias Model Testing Results:**

```
Classification report:
              precision    recall  f1-score   support

     NEUTRAL       1.00      1.00      1.00       456
     PRO_GOV       1.00      0.98      0.99        58
    ANTI_GOV       1.00      0.97      0.99        35

    accuracy                           1.00       549
   macro avg       1.00      0.98      0.99       549
weighted avg       1.00      1.00      1.00       549

Confusion matrix:
[[456   0   0]
 [  1  57   0]
 [  1   0  34]]

✅ Final model saved to: ./FINAL_XLMR_LARGE_GOV_STANCE\final_model

=== DONE ===
```

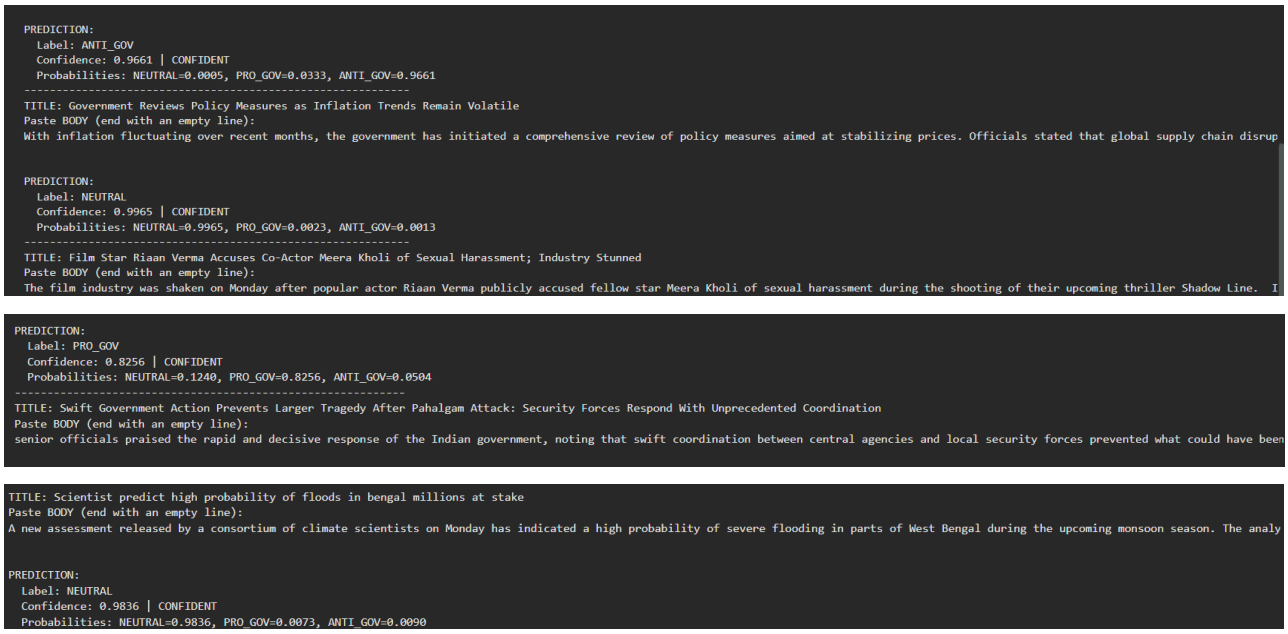


Figure 11 : Output Results for political bias model

5.5 Results and Discussions

The successful integration of web scraping, natural language processing (NLP) and machine learning (ML) into a coherent system to provide a comprehensive understanding of news articles from an array of Indian online publications has been demonstrated through the NewsInsights platform's implementation and testing. The platform was able to process real-time news articles from a number of different Indian media outlets. It converted unstructured text data into structure information that can be interpreted.

The primary finding from the analysis was the ability to utilize event-based clustering to locate news items about the same real-world-event. All of the news covering the same physical event were consistently cluster grouped together based on semantic embeddings regardless of which news publisher published the article. This resulted in the ability to conduct a comparative analysis of news from multiple sources and reduced redundancy when consuming news. Any minor differences in vocabulary or how the articles framed the news events did not have a significant negative impact on the cluster grouping of similar articles. This demonstrates the robustness of utilizing semantic embeddings when determining the similarity of two articles.

Additionally, the ability to accurately classify articles as to whether the articles contained bias either towards or against the Government, or were Neutral was successfully realized in the political bias detection module. By providing confidence levels as well as uncertainty flags, this allowed the system to provide less confidence in cases of borderline labeling. A probabilistic classification aligns well with the subjective nature of polarizing political opinions and provides a greater degree of transparency to users.

Long article summaries were effectively created from news articles using Artificial Intelligence (AI). Summaries maintained the same context and key information but reduced the overall amount of information provided to readers, thus reducing the effect of information overload.

Sentiment analysis adds a new dimension to the analysis of news articles. In addition to identifying a news article's stance, sentiment analysis also captures the emotional tone behind a story to provide an understanding of how a news narrative is constructed.

The recommendation engine produces recommendations based on two factors: (1) articles are recommended based upon their semantic relatedness to an individual user's reading history and (2) the user is exposed to multiple viewpoints.

The findings indicate that NewsInsights is a reliable, intelligent and bias-aware news aggregation tool that meets its information analytic and usability goals.

5.6 Inferences Drawn

The analysis of the implementation of NewsInsights yielded a plethora of insights and findings with regards to the efficiency, feasibility and utility of AI based systems for news analyses. One finding demonstrates that Semantic Event Clustering has been extraordinarily beneficial for the way in which news articles are organized. This feature of NewsInsights allows users to easily find and compare articles from various outlets that pertain to the same real world event, thereby reducing redundancy and improving the identification of the various framing differences and potentially biased language and other sources that report on the same event.

Secondly, the political stance classification model, which was implemented using a transformer based architecture, was successful in accurately determining the narrative orientation of Indian News Articles by classifying the articles into three categories (Pro Government, Anti Government and Neutral). It was determined that the model's confidence score, as well as uncertainty flags, indicate that political bias is not a binary or absolute phenomenon. Instead, it is important to view political bias as a spectrum, with each individual article having a different level of bias. This conclusion reinforces the need for transparency and the need for awareness of uncertainty when using NLP techniques in sensitive applications.

Thirdly, the use of AI-generated summaries and sentiment indicators significantly reduced the excessive amount of information a user would typically be exposed to, while still allowing the user to maintain their ability to understand the important context of the news article. With these two features, a user can understand the main points of a complex news article, without missing the important nuances of each individual article. Finally, the personalization of individual recommendations based on an embedding-based approach was

able to offer users with relevant recommendations without providing them with an echo chamber or limited view of the world.

Collectively, these observations indicate that integrating clustering, stance detection, summarization, and recommendation into a unified pipeline results in a more interpretable, balanced, and user-centric news consumption experience.

5.7 Validation of Objectives

The NewsInsights project has delivered results that confirm the objectives established in the initial design. A main objective of the project was to create an unbiased and transparent way to Aggregate news content. This objective has been achieved through multi-source news aggregating, as well as using labels to mark political stances. More importantly, users will now be able to understand how their content is ideologically framed rather than simply consume the content without thought.

Using a transformer-based stance-detection model, the project achieved its objectives of identifying and classifying Political Bias. The model classifies news articles into three main categories (Pro-Government, Anti-Government, and Neutral) with probability scores associated with the classification. As a result, users are provided with a responsible way of understanding what the political tendencies of news articles mean when reading the article content.

An important goal of this project was to create ways to alleviate the effects of information overload; AI-based Abstractive Summarization is an example of how this goal was achieved. AI-based Abstractive Summarization uses machine learning to create summaries of long articles with similar keyword parameters to provide a concise representation of the original article.

Additionally, Keyword Extraction and Sentiment Analysis successfully provided immediate cues regarding the Context of news reports, as well as the Emotional aspect associated with the news report. Consequently, all these components of the NewsInsights system provide a means for users to better understand the evolution of their political beliefs through news articles and information.

The project achieved its objective of creating a personalized news recommendation system through creating the ability to Search for Similarity between users and News Articles using embedding and similarity search techniques, thus ensuring that users receive news articles relevant to their interests without reinforcing a one-sided Narrative. The implementation of the NewsInsights system demonstrates that the project objectives were successfully achieved in a Logical and Technically Sound manner.

CONCLUSIONS AND FUTURE SCOPE

6.1 Conclusions

The development of NewsInsights demonstrates the potential of integrating artificial intelligence with real-time news aggregation to deliver a more transparent, personalized, and insightful news consumption experience. The system successfully implements efficient aggregation, AI-powered summarization, and a recommendation engine that adapts to user preferences, thereby addressing the challenges of information overload and biased reporting.

While modules such as political bias detection are still under progress, the current implementation already showcases the platform's value in enhancing user engagement, improving accessibility of information, and fostering informed decision-making. The project highlights how combining automation, natural language processing, and personalization techniques can transform the way users interact with digital news platforms.

6.2 Environmental, Economic and Social Benefits

1. Environmental Benefits: By promoting digital-first consumption of news through aggregation, the system reduces reliance on physical newspapers, thereby lowering paper usage and the associated carbon footprint of print and distribution.
2. Economic Benefits: The platform supports efficient access to diverse news sources in a single interface, reducing the cost and time of information gathering for users. Additionally, the underlying AI modules—such as recommendation and summarization—demonstrate potential for scalable deployment in media and content industries, offering opportunities for monetization and cost-effective operations.
3. Social Benefits: NewsInsights enhances informed decision-making by addressing political bias, highlighting key insights, and providing personalized recommendations. This contributes to combating misinformation, fostering transparency in media consumption, and empowering users to access balanced perspectives. Ultimately, it encourages democratization of information, making quality news accessible to a wider audience.

6.3 Reflection

Developing the NewsInsights Capstone project helped develop and improve my technical (such as programming), analytical (such as data analysis/analytics and statistics) and collaborative (i.e. working with others) skills.

One of the major goals of this project was to learn how concepts learned in the classroom about Machine Learning, NLP and Software Engineering (i.e. 'Real-World') translate into the Real World by creating a set of models that will detect Political Bias and Perform a

Sentiment Analysis. Designing and implementing these models emphasised how Data Quality, Model Evaluation and Ethical Considerations are integral parts of AI (Artificial Intelligence) technology/AI Applications.

Additionally, from a System Design aspect, this project created an opportunity for us to learn the process of architecting scalable and modular applications (by designing a full-stack application using the technologies and frameworks of Django, REST APIs, PostgresSQL and a current Modern Front End Framework) and develop several Best Practices (as an example: Separation of Concerns, Asynchronous Processing, and API-Driven Development). Also, the need for Performance Optimization and Cost-Aware Design (specifically) while deploying AI Models that require Computational Resources.

The NewsInsights Capstone Project focused on developing our research and problem solving skills, which allowed us to conduct an exhaustive Literature Survey which assisted us in finding the gaps that exist in the research and identify the location of our work among other similar offerings. Moreover, the iterative process of debugging, testing, and integration assisted us in building our capacity for resiliency and adaptability to real-world situations, including dealing with Unreliable Data Sources and Limitations of Models.

6.4 Future Work Plan

The current NewsInsights platform performs well in detecting Political Bias, providing Summaries and Personalizing news, but there remains an opportunity to enhance and extend the platform.

A primary opportunity to expand NewsInsights is through the introduction of news analysis that supports Multilingual and Cross-language analysis. This will enable an expanded and more diverse audience as a result of including an entire region and non-English language news sources.

Additionally, incorporating Multimodal analysis of news to detect bias in Text, Image, Headline, Caption and Video Type will provide a complete view of how all media forms together to create Media and Narrative Bias. Currently, those forms are analyzed separately. By incorporating this, we will gain a greater understanding of Media and Narrative Bias, as well as enable a greater ability to detect bias.

Additionally, the Product Recommendation (Recommendation Engine) has the potential to further enhance Personalization and Preserving Viewpoint Diversity through the use of Collaborative Filtering, and Real-time Feedback Loops.

Scalability is another opportunity from the Infrastructure perspective, as it can be enhanced through leveraging Cloud Native Deployment, Auto-scaling Infrastructure and Optimizing Inference Pipelines, thereby optimizing Operating Costs and increasing scalability.

7.1 Challenges Faced

The NewsInsights platform faced numerous technical and conceptual hurdles throughout its development process. Foremost among these hurdles was developing a method for classifying how a specific article presents information based on the level of political bias as perceived by the viewer. In the case of India, this was particularly difficult because of the subtleties of language and cultural context that led to sometimes large variances in framing across articles. Creating an effective stance detection model also relied heavily on effectively managing the degree of certainty given to each article and detecting when the model was going to yield overconfident or misleading results.

Another major hurdle was acquiring and processing the vast quantities of data collected through web scraping from very large news websites. My experience with web crawling allowed me to easily navigate dynamic webpages, inconsistent HTML formats, and delay-based restrictions in a highly efficient manner, but ensuring that this data was correct, contained no duplicates, and was temporally relevant made the data pipeline much more complex than originally anticipated.

Integrating multiple AI technologies (e.g., clustering, stance detection, summarization, sentiment analysis, and recommendation) into an integrated, asynchronous data pipeline created architectural challenges. Balancing computational costs, latency, scalability, and product time to reach consumers with giving that same consumer an easy-to-use, responsive experience is very complex, requiring extensive design with an emphasis on modularity and the appropriate use of background processing.

7.2 Relevant Subjects

The NewsInsights project encompasses several facets of the undergraduate studies, representing a high level of interdisciplinary integration. The project has employed essential principles related to Machine Learning and Artificial Intelligence to develop transformer-based (e.g., BERT) models for detecting political stance, performing sentiment analysis and summarizing. Further, fundamental principles from Natural Language Processing (NLP) played a critical role in the development of the NewsInsights project; including the creation of vector embeddings, text pre-processing, semantic similarity analysis and generating abstractions through the use of neural networks. Principles from Database Management Systems are represented in the creation of relational data schemas and the ability to perform efficient vector similarity searches through the PostgreSQL (with pgvector) database management system. Additionally, the application of Software Engineering concepts related to modular design, SDLC (Software Development Life Cycle) and API-based designs were utilized; and finally, the application of Web Technologies

principles including user experience design and front-end/back-end integration principles. This project represents a perfect example of how multiple areas of study are integrated into a functional, real-world solution.

7.3 Interdisciplinary Knowledge Sharing

To implement this project, concepts and theories from many different fall into the category of ‘outside computer science’. The basic ideas of political framing, narrative bias, and agenda-setting, extracted primarily through empirical approaches rooted in political science and media studies were able to directly shape and develop the algorithm that identifies sentiment in articles polled within the user’s feed. In terms of developing the framework for the algorithm, incorporating principles from ethical AI and its implications for designing systems that provide transparency, manage uncertainty in results, and avoid making definitive statements about the presence or absence of bias, proved to be valuable. Likewise, the HCI research that informed design decisions about how to display bias indicators, summary information, and sentiment labels to users, resulted in a system that does not overwhelm users. The interdisciplinary nature of the project helped the system to go beyond just providing an automated/technical classification of articles, to an analysis platform that used contextual information about those articles and the way that they may have aligned with social justice principles.

7.4 Peer Assessment Matrix

This project was created through collaboration among team members with individual responsibilities in designing the system, implementing the backend, integrating AI models, developing the frontend, and producing documentation. Frequent meetings, design

			Evaluation of			
		Manav Singhal	Sakaar Sen	Nandan Garg	Palak Mahajan	Ridham Uppal
	Manav Singhal	5	5	5	5	5
Evaluation By	Sakaar Sen	5	5	5	5	5
	Nandan Garg	5	5	5	5	5
	Palak Mahajan	5	5	5	5	5
	Ridham Uppal	5	5	5	5	5

evaluation meetings, and taking time to combine code allowed the team to have equal contributions and remain aligned with the project's objectives.

7.5 Role Playing and Work Schedule

The project's various components (Requirements, Design, Development, Testing, Documentation) were each assigned to multiple team members. Team member's roles were assigned dynamically based upon the complexity of the project tasks and individual team members' areas of Technical Expertise and Experience. Therefore, Project tasks were performed using a "Parallel Development" methodology.

The timeline of the project was separated into phases that mirrored the Work Breakdown Structure (WBS) of the entire project. The project moved through the phases: System Design Phase and the Data Collection Phase to the Integration of the Artificial Intelligence (AI) Model Phase and the Full Stack Development (FSD) Phase. The project maintained a consistent flow of progress based on Milestone Dates maintained throughout the life of the project.

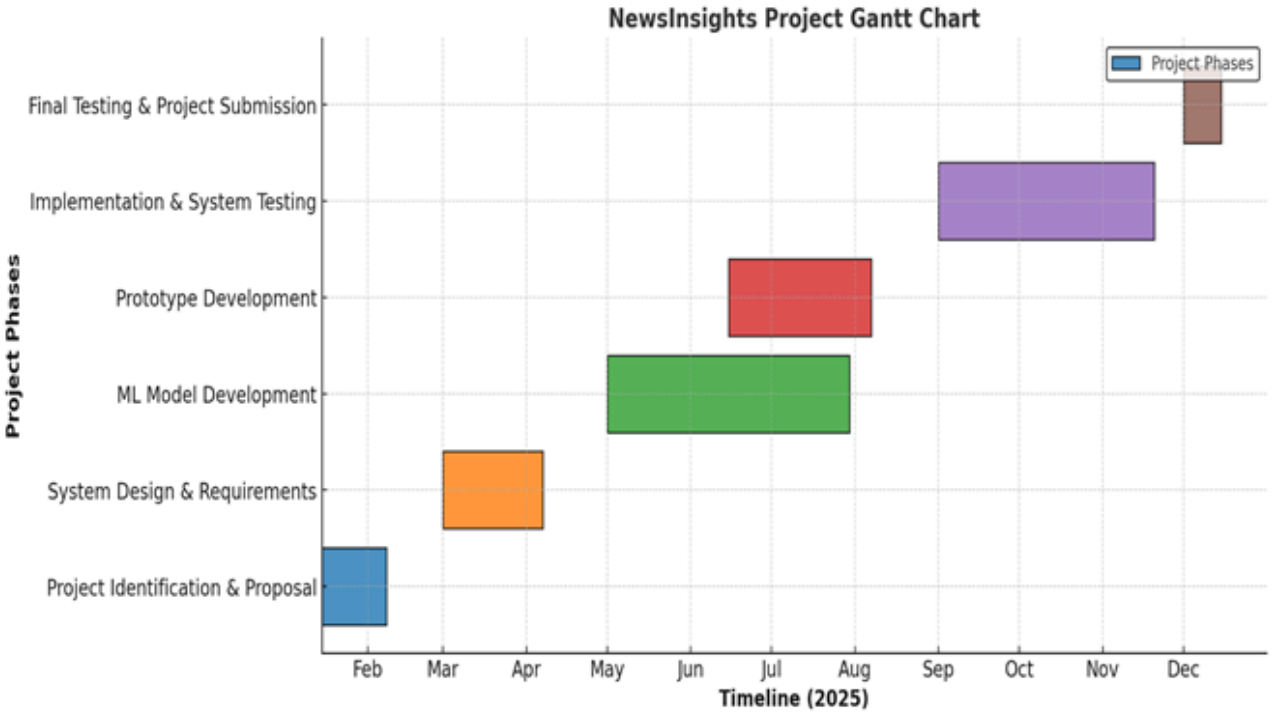


Figure 12 : Work Schedule Gantt Chart

Sakaar Sen has been involved in both backend development and the integration of various systems. He is responsible for creating and implementing Django-based, RESTful APIs with the Django Rest Framework. To facilitate working with Data on a Database level, he has Developed the Database Interaction (Data Layer) of the project. Additionally, he created a process for integrating asynchronous processing with the help of Celery and Redis. Sakaar has played a critical role in defining the overall **Back-end Architecture** of the Project.

Nandan Garg has supported **Machine Learning Development**, and he has played a role in the design and development of the various A.I Modules of the Project. He has further assisted in the process of integrating the models into the AI workflow through Model Development and Experiments, and optimising the performance of the AI Workflows.

Ridham Uppal has played a key role in the **Frontend development** of the project using Next.js. He has Developed the User Interface components for articles, including the summaries, Stance Labels, and Recommendation Views. He has also aided in the integration of Frontend and Backend development through the refinement of the usability.

Manav Singhal has created and participated in the development of both the **Natural Language Processing and Machine Learning** portions of the project. His key areas of focus were the creation of the Transformer- based models for Political Stance Detection, the Creation of the Self-Labeled Dataset (using End-User Feedback) with the assistance of the LLM, the analysis of Confidence and Uncertainty, and the Secure Integration of ML Inference into the Backend Pipeline. He has played an important role in Decisions around System Architecture and the Technical Documentation associated with the Various AI Modules.

Palak Mahajan – Worked on **frontend development** and documentation, contributing to UI layout consistency, report preparation, formatting, and integration of figures and diagrams. Also assisted in data preprocessing and organization.

7.6 Student Outcomes (SO) Description and Performance Indicators (PI)

SO No.	SO Description	Outcome Achieved in NewsInsights Project
1	Ability to identify and formulate problems related to computational domain	Identified the problem of political bias, information overload, and lack of transparency in digital news consumption. Formulated a computational solution using NLP and machine learning to detect stance, summarize content, and organize news effectively.
2	Apply engineering, science, and mathematics body of knowledge to obtain analytical, numerical, and statistical solutions	Applied mathematical concepts such as vector embeddings, cosine similarity, probability distributions (softmax), and confidence thresholds to clustering, recommendation, and political stance detection tasks.

3	Design computing system(s) to address needs in different problem domains and build prototypes	Designed and implemented a full-stack AI-powered news aggregation system integrating backend services, ML pipelines, databases, and frontend interfaces that meet defined functional requirements.
4	Ability to analyze the economic trade-offs in computing systems	Analyzed infrastructure and AI inference costs, balancing performance and operational expenses through selective self-hosting of models and use of external inference endpoints for resource-intensive tasks.
5	Prepare and present variety of documents according to computing standards and protocols	Prepared comprehensive project documentation following IEEE standards, including problem analysis, methodology, system design, and evaluation.
6	Communicate effectively with peers using adequate technical knowledge	Demonstrated effective technical communication through collaborative design discussions, code integration, documentation, and coordinated development across backend, frontend, and AI components.
7	Aware of ethical and professional responsibilities while designing computing solutions	Addressed ethical considerations in AI by ensuring transparency in political stance detection, incorporating uncertainty handling, and avoiding absolute or misleading bias claims.
8	Evaluate computational solutions considering environmental, societal, and economic contexts	Evaluated the societal impact of media bias and misinformation, designing the system to promote media literacy, balanced viewpoints, and responsible information consumption.
9	Participate in development and selection of ideas to meet objectives	Actively participated in ideation, design decisions, and feature selection to ensure alignment with project objectives such as bias awareness, summarization, and personalization.
10	Plan, share, and execute responsibilities to function effectively in a team	Demonstrated teamwork through role-based task allocation, coordinated development schedules, and shared ownership of system integration and documentation.
11	Ability to perform experimentations and analyze obtained results	Performed system-level experimentation by analyzing clustering behavior, stance classification confidence, and summarization effectiveness under real-world data conditions.

12	Ability to analyze and interpret data, make necessary judgments and conclusions	Interpreted model outputs, probability scores, and uncertainty flags to draw meaningful conclusions about political framing and system reliability.
13	Ability to explore and utilize resources to enhance self-learning	Independently explored advanced resources such as transformer models, Hugging Face libraries, LLM-assisted labeling, and large-scale NLP research to enhance system capability.

7.7 Brief Analytical Assessment

Analyzed from a technical standpoint, NewsInsights is a good example of how state-of-the-art natural language processing (NLP) and machine learning techniques are being used to address socially relevant issues. The project shows how to balance the technical rigor involved with applying state-of-the-art technologies to the problems of society with the ethical and responsible ways in which they can be used through its integration of multi-source comparison, uncertainty-aware stance detection and explainable insights. In addition, the project uses a modular architecture and scalable design that provide a solid foundation for future growth opportunities. Furthermore, the interdisciplinary framing of this project provides both academic and practical relevancy to the work being done via NewsInsights. The project successfully fulfilled its intended learning objectives and provided a coherent and meaningful solution to the current issues in digital news consumption for the students who were involved in its development as a capstone project.

REFERENCES

[1] Media bias – Wikipedia, accessed Aug 21, 2025,
https://en.wikipedia.org/wiki/Media_bias

[2] Types of Media Bias – McGill-Toolen, accessed Aug 21, 2025,

https://www.mcgill-toolen.org/ourpages/users/tenhunm/ap_us_gov/Chapter%2012%20The%20Media/Types%20of%20media%20bias.pdf

[3] Detecting and Mitigating Bias in NLP – Brookings, accessed Aug 21, 2025,
https://www.brookings.edu/articles/detecting-and-mitigating-bias-in-natural-language-processing/

[4] Political News Bias Detection – Inference of Media Bias and Content Quality Using NLP, accessed Dec 1, 2022,
https://arxiv.org/pdf/2212.00237

[5] Sun et al. (2025). PRISM Framework – ACL 2025, accessed Aug 21, 2025,
https://aclanthology.org/2025.acl-long.1344.pdf

[6] Maab et al. (2023). Target-Aware Contextual Political Bias Detection – IJCNLP 2023, accessed Aug 21, 2025,
https://aclanthology.org/2023.ijcnlp-main.50

[7] Sar & Roy (2024). Navigating Nuance with LLMs – JCDL 2024, accessed Aug 21, 2025,
https://arxiv.org/abs/2501.00782

[8] Unpacking Political Bias in LLMs – arXiv:2412.16746, accessed Aug 21, 2025,
https://arxiv.org/abs/2412.16746

[9] Bias in Text Analysis for International Relations – Oxford Academic, accessed Aug 21, 2025,
https://academic.oup.com/isagsq/article/2/3/ksac021/6584769

[10] Brookings (2023). Bias in NLP Models, accessed Aug 21, 2025,
https://www.brookings.edu/articles/detecting-and-mitigating-bias-in-natural-language-processing/

[11] BASIL, BABE, MBIB datasets; AllSides, BigNews, NewsSpectrum corpora, accessed Aug 21, 2025,
https://aclanthology.org/venues/ (general dataset references)

[12] Hugging Face Transformers, PyTorch/TensorFlow libraries, accessed Aug 21, 2025,
https://huggingface.co/transformers/
https://pytorch.org/
https://www.tensorflow.org/

[13]:Bias in Text Analysis for International Relations Research - Oxford Academic, accessed August 21, 2025, **https://academic.oup.com/isagsq/article/2/3/ksac021/6584769**

[14]: arxiv.org, accessed August 21, 2025, **https://arxiv.org/html/2501.00782v1**

[15]:Detecting Bias in the Media, accessed August 21, 2025, **https://www.edu.gov.mb.ca/k12/cur/socstud/foundation_gr9/blms/9-1-3g.pdf**

Plagiarism Report

ORIGINALITY REPORT

6%	4%	1%	5%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Thapar University, Patiala Student Paper	3%
2	Submitted to Indian Institute of Technology Roorkee Student Paper	1%
3	Submitted to Thapar Institute Of Engineering and Technology, Patiala Student Paper	<1%
4	aclanthology.org Internet Source	<1%
5	www.mdpi.com Internet Source	<1%
6	www.coursehero.com Internet Source	<1%
7	www.diva-portal.org Internet Source	<1%
8	Submitted to Far Eastern University Student Paper	<1%
9	download.bibis.ir Internet Source	<1%
10	pubmed.ncbi.nlm.nih.gov Internet Source	<1%
11	Submitted to Swinburne University of Technology Student Paper	<1%
12	journalofbigdata.springeropen.com Internet Source	<1%

13	Submitted to Mahatma Gandhi Central University, Motihari, Bihar Student Paper	<1%
14	Submitted to Fort Valley State Univeristy Student Paper	<1%
15	baou.edu.in Internet Source	<1%
16	pure.rug.nl Internet Source	<1%
17	paperswithcode.com Internet Source	<1%
18	Submitted to University of Dundee Student Paper	<1%
19	Submitted to University of Wales Institute, Cardiff Student Paper	<1%
20	ijsrem.com Internet Source	<1%
21	wetlands.msuextension.org Internet Source	<1%
22	Koyel Ghosh, Apurbalal Senapati. "Hate speech detection in low-resourced Indian languages: An analysis of transformer-based monolingual and multilingual models with cross-lingual experiments", Natural Language Processing, 2024 Publication	<1%
23	rifl.unical.it Internet Source	<1%
24	repository.rice.edu Internet Source	<1%
25	www.spglobal.com Internet Source	<1%

26	arxiv.org Internet Source	<1%
27	dl.ucsc.cmb.ac.lk Internet Source	<1%
28	www.arxiv.org Internet Source	<1%
29	Ankita Bansal, Abha Jain, Sarika Jain, Vishal Jain, Ankur Choudhary. "Computational Intelligence Techniques and Their Applications to Software Engineering Problems", CRC Press, 2020 Publication	<1%
30	Submitted to Heriot-Watt University Student Paper	<1%
31	Sukhpreet Kaur, Amanpreet Kaur, Manish Kumar. "Recent Advances in Computational Methods in Science and Technology - Volume 2", CRC Press, 2026 Publication	<1%
32	dokumen.pub Internet Source	<1%
33	www.termpaperwarehouse.com Internet Source	<1%
34	www.websiteperu.com Internet Source	<1%
35	Tobias Oberdieck, Peter Schlecht. "chapter 10 Rethinking Management Research and Education", IGI Global, 2025 Publication	<1%
36	Vijay Kumar Sharma, Sachin Kumar Gupta. "High-Performance Automation Methods for Computational Intelligent Systems - Challenges, Opportunities, and Applications", CRC Press, 2025	<1%
37	en.wikipedia.org Internet Source	<1%
38	"Information Technology and Systems", Springer Science and Business Media LLC, 2025 Publication	<1%